



Univerzitet u Nišu  
Elektronski fakultet u Nišu



## Replikacija i konzistentnost kod Cassandra

Seminarski rad

Mentor:  
Prof. dr Aleksandar Stanimirović

Student:  
Luka Veličković, 2101

Niš, Jun 2026.

# Sadržaj

|                                                 |    |
|-------------------------------------------------|----|
| 1 Uvod.....                                     | 4  |
| 2 Cassandra u svetu distribuiranih sistema..... | 5  |
| 2.1 Rack i data centner.....                    | 5  |
| 2.2 Particionisanje podataka.....               | 5  |
| 2.2.1 Rebalansiranje particija.....             | 6  |
| 2.3 Gossip protocol.....                        | 7  |
| 2.3.1 Detekcija otkaza.....                     | 8  |
| 2.4 CAP teorema i Cassandra.....                | 8  |
| 3 Replikacija kod Cassandre.....                | 10 |
| 3.1 Token Ring.....                             | 10 |
| 3.2 vnodes.....                                 | 11 |
| 3.3 Replikacija.....                            | 12 |
| 3.3.1 Single-leader replikacija.....            | 12 |
| 3.3.2 Multi-leader replikacija.....             | 13 |
| 3.3.3 Leaderless replikacija.....               | 14 |
| 3.4 Leaderless replikacija u Cassandri.....     | 15 |
| 3.4.1 Strategije replikacije.....               | 15 |
| 3.4.1.1 SimpleStrategy.....                     | 15 |
| 3.4.1.2 NetworkTopologyStrategy.....            | 16 |
| 3.4.1.3 Transient replication.....              | 17 |
| 3.4.1.4 Sinhronizacija replika.....             | 17 |
| 4 Konzistentnost kod Cassandre.....             | 18 |
| 4.1 Konzistentnost.....                         | 18 |
| 4.1.1 Čitaj svoje upise.....                    | 18 |
| 4.1.2 Monotona čitanja.....                     | 18 |
| 4.1.3 Consistent prefix reads.....              | 19 |
| 4.2 Kvorum.....                                 | 20 |
| 4.2.1 Nivoi konzistentnosti kod Cassandre.....  | 20 |
| 4.2.2 Lightweight transakcije i Paxos.....      | 20 |
| 4.3 Koordinator.....                            | 21 |
| 4.4 Sloppy kvorum i hinted handoff.....         | 22 |
| 4.5 Repair.....                                 | 23 |
| 4.6 Read repair.....                            | 23 |
| 4.7 Read (repair) path.....                     | 23 |
| 4.8 Konkurentni upisi.....                      | 24 |
| 5 Praktični prikaz.....                         | 26 |
| 5.1 Cassandra cluster sa tri čvora.....         | 26 |
| 5.2 Faktor replikacije 1.....                   | 26 |
| 5.3 Faktor replikacije 2.....                   | 27 |
| 5.3.1 Hinted handoff.....                       | 28 |
| 5.4 Nivoi konzistencije.....                    | 29 |
| 5.5 Lightweight transakcije.....                | 31 |
| 5.6 Anty-entropy repair.....                    | 32 |
| 6 Literatura.....                               | 33 |



# 1 Uvod

Perzistentno skladištenje podataka predstavlja neizostavni deo bilo koje savremene aplikacije. Sa razvojem interneta, masovnim porastom broja korisnika i eksplozijom količine podataka generisanih u sekundi, tradicionalni pristup skladištenju na jednom centralizovanom serveru naišao je na svoja fizička ograničenja. Dok su monolitne relacije baze podataka decenijama uspešno obezbeđivale ACID svojstva, zahtevi modernih globalnih platformi – u pogledu konstantne dostupnosti (24/7), niske latencije i horizontalnog skaliranja – nametnuli su potrebu za drugačijim pristupom. Odgovor na ove izazove pronađen je u razvoju distribuiranih sistema i NoSQL baza podataka.

U osnovi, distribuirana baza podataka predstavlja kolekciju čvorova (servera) povezanih mrežom, koji zajednički deluju kako bi korisniku pružili iluziju jedinstvenog sistema. Korisniku baza podataka obezbeđuje nivo apstrakcije koji dozvoljava upravljanje podacima preko deklarativnih interfejsa, poput CQL-a (Cassandra Query Language), krijući iza jednostavnog sintaksnog interfejsa izuzetno kompleksnu logiku distribucije, mrežne komunikacije i koordinacije među čvorovima. Kao i kod centralizovanih sistema, od korisnika se ne zahteva duboko poznavanje unutrašnjeg funkcionisanja baze podataka da bi otpočeo rad sa njom. Međutim, distribuirani sistemi nisu magične kutije; oni su ograničeni fundamentalnim zakonima distribuisanog računarstva, pre svega CAP teoremom, i efikasni su onoliko koliko je efikasan i inženjer koji razume kompromise koje ovi sistemi prave u pozadini.

U centru arhitekture Cassandra baze podataka nalazi se decentralizovani model bez centralne tačke otkaza (masterless/leaderless), gde je svaki čvor u klasteru ravnopravan. Za razliku od sistema sa jednim liderom, Cassandra optimizuje podsistem za prihvatanje podataka tako što omogućava da bilo koji čvor primi upis ili čitanje, raspoređujući podatke duž kružne strukture poznate kao *Token Ring*. Kako bi se obezbedila otpornost na hardverske otkaze i mrežne particije, Cassandra se oslanja na napredne mehanizme replikacije, asinhronu komunikaciju putem *Gossip* protokola, i privremeno skladištenje propuštenih upisa u obliku *Hinted Handoff* zapisa. [1]

Najveća snaga, ali ujedno i najveći izazov kod Cassandre, ogleda se u konceptu prilagodljive konzistentnosti (*Tunable Consistency*). Cassandra omogućava korisniku da za svaku pojedinačnu operaciju balansira između brzine odziva (latencije) i nivoa konzistentnosti podataka kroz nivoe kao što su ONE, QUORUM ili ALL. Kada su u pitanju kritične operacije koje zahtevaju strogu linearnost, Cassandra implementira takozvane lagane transakcije (*Lightweight Transactions - LWT*) zasnovane na Paxos konsenzus protokolu. Svi ovi mehanizmi konstantno rade na održavanju eventualne konzistentnosti (*Eventual Consistency*), dopunjeni pozadinskim procesima aktivne popravke podataka, poput *Read Repair-a* i manualnog *Anti-Entropy Repair-a* zasnovanog na Merkleovim kriptografskim stablima. [1]

## 2 Cassandra u svetu distribuiranih sistema

Osnovna karakteristika Cassandre je da se radi o distribuiranoj bazi podataka, koja se izvršava na više mašina, pri čemu korisnici baze ceo sistem vide kao jedinstven. Svaka mašina na kojoj se izvršava Cassandra naziva se čvor, a skup svih čvorova naziva se *cluster*. Pored toga, *Cassandra* je decentralizovana, što znači da su svi čvorovi jednaki, odnosno ne postoji ni jedan čvor koji ima zaduženja koja nijedan drugi čvor ne može da ima. Komunikacija između čvorova je zasnovana na *peer-to-peer* modelu. Pored toga, za Cassandru se kaže da ima elastičnu skalabilnost, što znači da može vrlo lako da horizontalno skalira, dodavanjem novih čvorova u cluster, odnosno da se smanji broj čvorova uklanjanjem čvorova iz cluster-a u zavisnosti od opterećenja sistema. Pored toga za Cassandru još važi da nudi *high availability* i *fault tolerance* i *tunable consistency*, o čemu će detaljnije biti reči u nastavku rada. [1]

### 2.1 Rack i data centner

Jedan od osnovnih slučajeva kada se Cassandra koristi je u sistemima koji se prostiru na različitim fizičkim lokacijama (npr. kako bi se sistem geografski približio krajnjim korisnicima). Radi grupisanja čvorova koji pripadaju jednom cluster-u, definišu se dva nivoa – *rack* i *data centar*. *Rack* je logički set čvorova koji su bliski međusobno, potencijalno mašine u jednom fizičkom *rack-u*. *Data centar* je logički set *rack-ova*, koji mogu da se nalaze u istoj objektu i povezani su pouzdanom mrežom.

### 2.2 Partitionisanje podataka

Kako je Cassandra prvenstveno distribuirana baza, to znači da prilikom upisa i čitanja treba doneti odluku koji od čvorova u cluster-u kontaktirati. Postoji mogućnost da se zahtev pošalje do svih čvorova, ali na taj način se gubi poenta da se ceo cluster posmatra kao jedinstvena baza podataka. Partitionisanje je tehnika koja omogućava da se podaci podele u tzv. particije, particije se dodele čvorovima nakon čega se čvoru pristupa u slučaju pristupa određenom podatku. Ovde u obzir ne uzima replikacija podataka, kojoj je kasnije u radu posvećeno poglavlje. Za potrebe ovog poglavlja pretpostavljamo da se jedan podatak čuva samo na jednom čvoru.

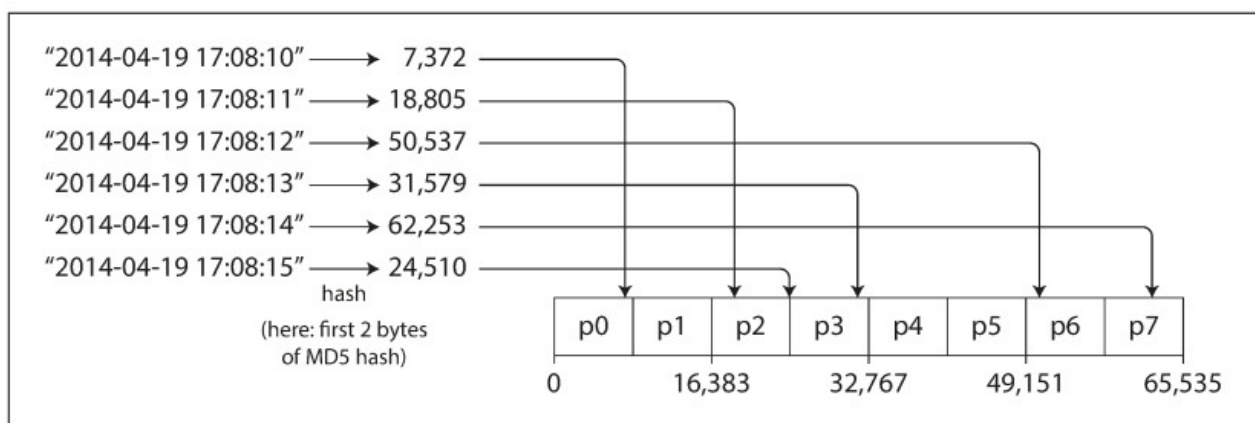
Partitionisanje se može sresti pod različitim imenima – *shard* kod MongoDB-a i Elasticsearch-a, *region* kod Hbase-a, *tablet* kod BigTable-a, *vnode* kod Cassandre i Riak-a. Particije se definišu tako da svaki podatak pripada tačno jednoj particiji. Glavni razlog za partitionisanje podataka je skalabilnost. Različite particije mogu se smeštati na različitim čvorovima u sistemu u *shared-nothing* arhitekturi. [3]

Kao što je pomenuto, osnovni cilj partitionisanja je skalabilnost, odnosno mogućnost da se opterećenje ravnomerno rasporedi po čvorovima. Jedan od problema koji se može javiti kod partitionisanja podataka je neravnomerno raspoređivanje podataka po čvorovima, odnosno *skew*. Kao posledica toga javlja se usko grlo kod nekog (-ih) čvorova kojima se češće pristupa nego ostalima. Jedno od mogućih rešenja ovog problema je da se podaci nasumično raspoređuju po čvorovima, ali se kod tog pristupa javlja problem prilikom čitanja podataka.

Druga mogućnost je partitionisanje po intervalima. Svakoj particiji se dodeljuje određeni interval i svi ključevi koji pripadaju tom intervalu se smeštaju u tu particiju. Dodatno, opsezi koje obuhvataju dve particije ne moraju da budu jednaki, jedan opseg potencijalno može da obuhvati

veći broj ključeva koji se ređe javljaju od drugog opsega. Ovakav pristup se koristi kod BigTable-a i Hbase-a. Jedan od problema kod ovakvog pristupa je da i dalje mogu da se jave uska grla, kada se na primer kao ključ koristi *timestamp*. U tom slučaju particija kojoj pripadaju vremenski trenuci za taj dan obrađuje sve zahteve. [3]

Treća mogućnost, ujedno i ona koju koristi Cassandra, je particionisanje po hešu ključa. Ideja je da se heširanjem ključa izbegnu uska grla. U tu svrhu Cassandra koristi MD5 algoritam za heširanje. Svakoj particiji se dodeljuje opseg heševa i na osnovu izračunatog heša određuje se gde se podatak smešta. Particionisanje na osnovu heša je prikazano na slici 1.



Slika 1- Particionisanje na osnovu heša ključa, preuzeto iz [3]

Nedostaka kod ovog pristupa je da se gubi mogućnost da se izvršavaju upiti po opsegu, obzirom da dva ključa koja su u poretku jedan pored drugog, nakon heširanja, mogu da završe u različitim particijama. Cassandra zapravo pravi kompromis između pomenute dve strategije, tako što nudi mogućnost kompozitnog primarnog ključa. Primarni ključ se sastoji od dela koji se koristi kako bi se odredila particija u kojoj se nalazi podatak (tzv. *partitioning key*) i dela koji definiše poredak ključeva u toj particiji (tzv. *clustering key*). [3]

### 2.2.1 Rebalansiranje particija

Rebalansiranje predstavlja postupak kojim se podaci razmenjuju između čvorova u clusteru iz nekog razloga. Razlog može da bude dodavanje novog čvora usled povećanja opterećenja, pad mašine u cluster-u itd. Minimalna očekivanja koja rebalansiranje treba da ispuni su:

- Nakon rebalansiranja, podaci treba da budu ravnomerno raspoređeni između čvorova
- Dok se dešava rebalansiranje baza podatak treba da nastavi da radi nesmetano
- Treba minimizirati količinu podataka koji se razmenjuju, obzirom da razmena zahteva mrežnu aktivnost kao i disk IO

Postoji nekoliko strategija za rebalansiranje. Najjednostavnija je korišćenje modulo (mod) operatora. Za svaki ključ (ili heš ključa) se nalazi ostatak pri deljenju sa, na primer, brojem particija, i dobijeni broj određuje novu particiju. Ovaj način dovodi do pomeranja ogromne količine podataka između particija.

Postoje pristupi koji se koriste kada sistem koristi fiksni broj particija i oni se svode na to da se prilikom dodavanja/uklanjanja čvora iz clustera, mali broj celih particija pomera između čvorova. Ovaj pristup se koristi kod Riak-a, Elasticsearch-a, Voldemort-a... Druga mogućnost je dinamičko particionisanje, koje koristi Hbase. Kada particija preraste određenu definisanu veličinu, dešava se

podela (*split*) particije na 2 manje. Suprotno, kada se particija isprazni ispod određene granice, može se spojiti (*merge*) sa drugom particijom.

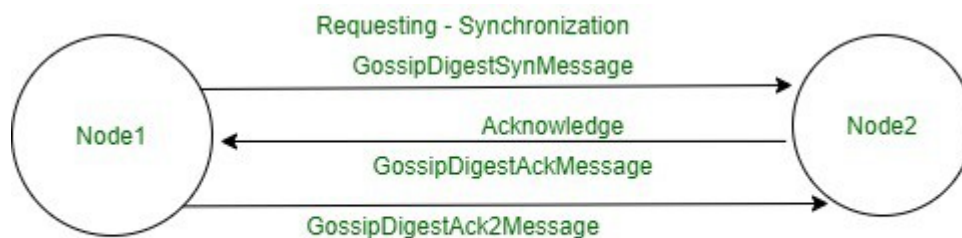
Cassandra ne koristi pomenuta dva pristupa za rebalansiranje, već koristi tehniku particionisanja proporcionalno broju čvorova. Kod dinamičkog particionisanja broj particija proporcionalan je količini podataka koji se skladište. Kod fiksnog broja particija, veličina particije proporcionalna je količini podataka koji se skladište. U oba slučaja je broj particija nezavisan od broja čvorova. Cassandra koristi pristup po kome je broj particija proporcionalan broju čvorova, odnosno drugim rečima postoji fiksni broj particija po čvoru. U ovom slučaju, veličina svake particije raste proporcionalno količini podataka, pri čemu broj čvorova ostaje nepromenjen. Kada se doda novi čvor u cluster, on nasumično bira fiksni broj postojećih particija koje se dele i postaje vlasnik polovine tih podataka. [3]

## 2.3 Gossip protocol

Replikacija, o kojoj će biti reči kasnije, kao i detekcija otkaza u clusteru oslanja se na tzv. *gossip protocol*. Korišćenjem ovog protokola Cassandra razmenjuje informacije između čvorova prilikom pokretanja cluster-a, kao što su verzije protokola, pripadnost članova. Kod ovog protokola, osim informacija o sopstevnom stanju, čvorovi razmenjuju i informacije koje znaju o drugim čvorovima. Svaka razmenjena informacija je verzionisana korišćenjem vektorskog časovnika (*vector clock*) koji je uređen par (*generation, version*), gde je *generation* monotoni *timestamp*, a *version* logički časovnik koji se inkrementira svake sekunde. Ovakav vektorski časovnik omogućava Cassandri da ignoriše stare informacije samo na osnovu informacija iz vektorskog časovnika. [2]

Gossip protocol se kod Cassandre izvršava na svim čvorovima nezavisno od drugih čvorova i periodično, približno svake sekunde. Aktivnosti koje se dešavaju su sledeće:

- Ažuriranje lokalnog vektorskog časovnika i formiranje lokalnog pogleda na cluster
- Odabir nasumičnog čvora u cluster sa kojim razmenjuje poruke
- Probabilistički pokušava da kontaktira pali čvor (ako postoji)
- Razmena poruke sa „seed“ čvorom, ako se to nije dogodilo u koraku 2.



**3 way communication using Gossip protocol**

Slika 2- Gossip poruke koje se razmenjuju između dva čvora u clusteru

Prilikom podizanja Cassandra cluster-a, jedan čvor se proglašava „seed“ čvorom. Bilo koji čvor može da bude seed čvor, a jedina razlika između „seed“ i „non-seed“ čvora je da je „seed“ čvorovima dozvoljeno da se priključe clusteru bez kontaktiranja drugog „seed“ čvora. Pored toga, tokom rada Cassandre, „seed“ čvorovi su *hotspot* za „tračarenje“ između čvorova. [2]

U aktuelnoj verziji Cassandre, gossip protokol se koristi i za propagiranje metapodataka i informacije o šemi. Ako neki čvor detektuje neslaganje između verzija šeme koje se koriste, zakazuju se šema sync poruke sa ostalim čvorovima.

### 2.3.1 Detekcija otkaza

Gossip predstavlja osnovu komunikacije u Cassandri, ali detektor otkaza je onaj koji ima završnu reč o tome da li će se neki čvor smatrati živim ili ne. Cassandra koristi varijantu *Phi Acural Failure Detector-a* i svaki čvor nezavisno za sebe donosi odluku o tome da li je neki drugi čvor živ. Ova odluka se najviše zasniva na primljenim *heartbeat* porukama. Kada detektor proceni da neki čvor možda nije živ, taj čvor koji izvršava taj detektor će prestati da rutira čitanja do tog čvora, a upisi će biti rutirani do *hint-ova*. O rutiranju zahteva za čitanje i upis biće više reči u narednim poglavljima. Kada čvor ponovo počne sa radom, šalje *heartbeat* poruku, a čvorovi nakon toga pokušavaju da ponovo ostvare konekciju sa tim čvorom. [2]

## 2.4 CAP teorema i Cassandra

Standardno tumačenje CAP teoreme tvrdi da distribuirani sistem može da ispuni samo dve od tri željene karakteristike – konzistentnost (*consistency*), dostupnost (*availability*) i *partition tolerance*.

- Consistency znači da svi klijenti vide iste podatke u isto vreme, bez obzira na to kom čvoru u distribuiranom sistemu pristupaju. Kako bi se ovo bilo moguće, prilikom upisa podataka na jedan čvor, podatak mora odmah biti prosleđen do svih drugih čvorova u sistemu pre nego što se upis proglasi uspešnim.
- Availability je osobina koja obezbeđuje da bilo koji zahtev za pristup podacima dobije odgovor, bez obzira na to da li su jedan ili više čvorova u distribuiranom sistemu pali.
- Partition tolerance se odnosi na komunikaciju između čvorova u distribuiranom sistemu – nedostupna ili privremeno ometena komunikacija između čvorova. Ova osobina znači da cluster mora da nastavi da radi bez obzira na te privremeno nedostupne veze između čvorova.

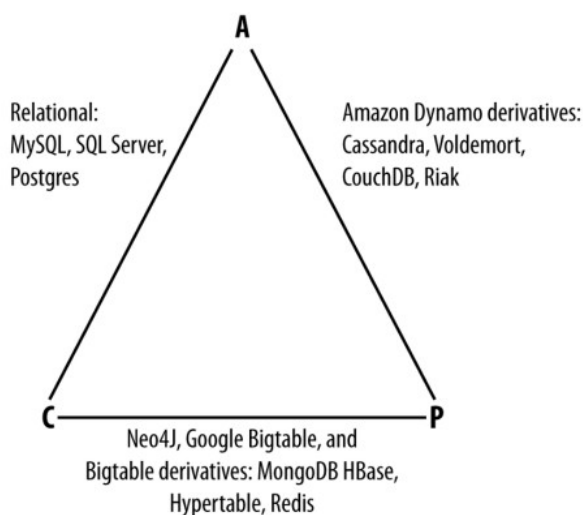
Kako CAP teorema tvrdi da distribuiran sistem može da ispuni samo dve od tri karakteristike, mogućnosti, u svetu distribuiranih baza podataka, su:

- CP – baza podataka koja ima CP osobine obezbeđuje consistency i partition tolerance, na račun dostupnosti. To znači da kada dođe do problema u mreži, sistem mora da ugasi nekonzistentan čvor dok se mrežna particija ne reši.
- AP – baze podataka koje donose AP nude dostupnost i otpornost na particionisanje mreže, na račun konzistentnosti. Kada dođe do problema u mreži, svi čvorovi ostaju dostupni, ali pojedini čvorovi mogu vraćati pogrešne (starije) vrednosti podataka. Kada se problem reši, čvorovi se obično resinhronizuju.
- CA – ovakav tip baze podataka postoji samo u teoriji, a potpuno upotrebljiva CA distribuirana baza podataka ne može da postoji. U teoriji, ova baza obezbeđuje konzistentnost i dostupnost kroz sve čvorove u distribuiranom sistemu. Međutim, ovo je neizvodljivo ukoliko postoji problem u mreži, a kako su mrežni problemi neizbežni, postizanje CA nije moguće. [4] Često se u kategoriju „CA“ svrstavaju relacione baze



podataka i njihova sposobnost da se podaci repliciraju sa glavne baze podataka na replike, pri čemu rad sistema staje ukoliko se detektuju problemi. [1]

Na slici 3 prikazana je raspodela pojedinih baza podataka u odnosu na karakteristike koje ispunjavaju.



Slika 3- Raspodela baza podataka u odnosu na CAP osobine koje zadovoljavaju

Kao što se sa slike 3 može primetiti, u odnosu na CAP teoremu, Cassandra je AP baza podataka. Ona nudi dostupnost i partition tolerance, ali ne može da garantuje konzistentnost sve vreme. Pošto kod Cassandre ne postoji glavni (master) čvor, svi čvorovi moraju konstantno međusobno da komuniciraju. To znači da realno, nekonzistentnost traje vrlo kratko. [1]

Originalni autor CAP teoreme Eric Brewer je 2012 objavio novu verziju teoreme, odnosno pregled/reviziju originalne teoreme. U tom radu Brewer tvrdi da je „samo 2 od 3“ obmanjujuća formulacija, obzirom da se availability i consistency žrtvuju samo u pristustvu partition tolerance-a. Otkrića u otklanjanju i oporavku od partition tolerance-a su napredovala što omogućava da se postigne izuzetno visok nivo i dostupnosti i konzistentnosti. Cassandra ovo postiže korišćenjem tehnika kao što su *hinted handoff* i *read repair* o kojima će više reči biti u narednim poglavljima. [1, 3]

## 3 Replikacija kod Cassandre

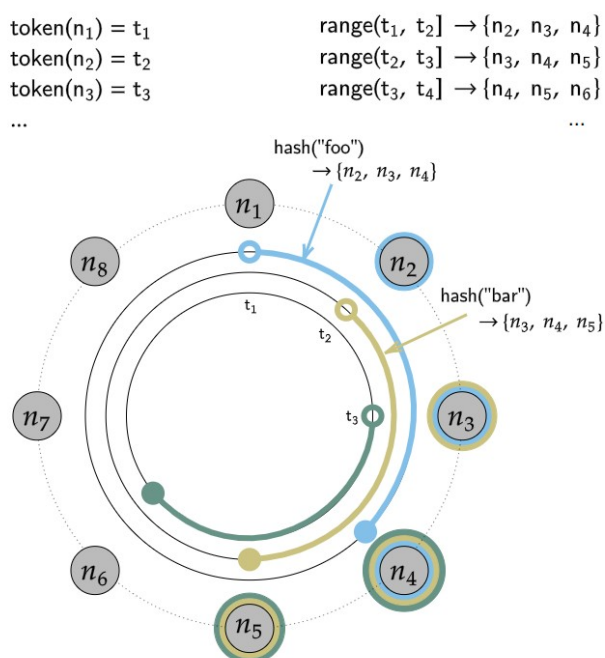
### 3.1 Token Ring

U prethodnom poglavlju, između ostalog, pomenuto je rebalansiranje particija, ali su finiji detalji ispušteni pa će se slika upotpuniti u ovom poglavlju.

Cassandra mapira svaki čvor na jedan ili više tokena na kontinualnom heš prstenu (hash ring) a vlasništvo nad ključem se definiše tako što se nakon heširanja ključa ovaj prsten obilazi u jednom smeru. [3] Cassandra ovaj prsten još naziva i consistent-hashing ring. Consistent hashing je tehnika koja se koristi kako bi prilikom promene veličine heš tabele (ili u slučaju distribuiranih sistema prilikom promene broja čvorova/particija) bilo potrebno remapiranje (odnosno premeštanje) samo  $n/m$  ključeva, gde je  $n$  ukupan broj ključeva, a  $m$  broj slotova. [5]

Pomenuti token je broj na heš prstenu. Svaki ključ podatka se kod Cassandre hešira u token i token definiše kom čvoru podatak pripada. Svaki čvor je vlasnik dela prstena, odnosno svih tokena od prethodnog tokena koji pripada drugom čvoru i svog tokena. Kada Cassandra želi da upiše ili pročita podatak, najpre hešira ključ u token, zatim nalazi poziciju tog ključa na prstenu, a nakon toga ide u smeru kazaljke na satu do pojave prvog tokena koji pripada nekom čvoru. Taj čvor se koristi za upis ili čitanje podatka.

Na primer ukoliko imamo cluster sa 8 čvorova, čiji su tokeni jednako udaljeni i faktor replikacije 3 (detaljnije o faktoru replikacije nešto kasnije). U tom slučaju, kako bismo pronašli sve čvorove koji sadrže traženi podatak, najpre heširamo ključ kako bismo dobili token, a nakon toga obilazimo prsten dok ne pronađemo tri jedinstvena čvora, nakon čega smo pronašli sve čvorove koji sadrže taj podatak. Ilustracija opisanog prikazana je na slici 4. Token koji pripada čvoru  $n_1$  je  $t_1$ , čvoru  $n_2$ ,  $t_2$ , a čvoru  $n_3$ ,  $t_3$ . To znači da je opseg tokena koji pripada čvoru  $n_2$  opseg definisan kao  $(t_1, t_2]$ , a na osnovu opisanog podaci iz tog opsega se još nalaze na čvorovima  $n_3$  i  $n_4$ , obzirom da su to prva tri čvora koja se nalaze kada se iz ovog opsega prsten obilazi u smeru kazaljke na satu.

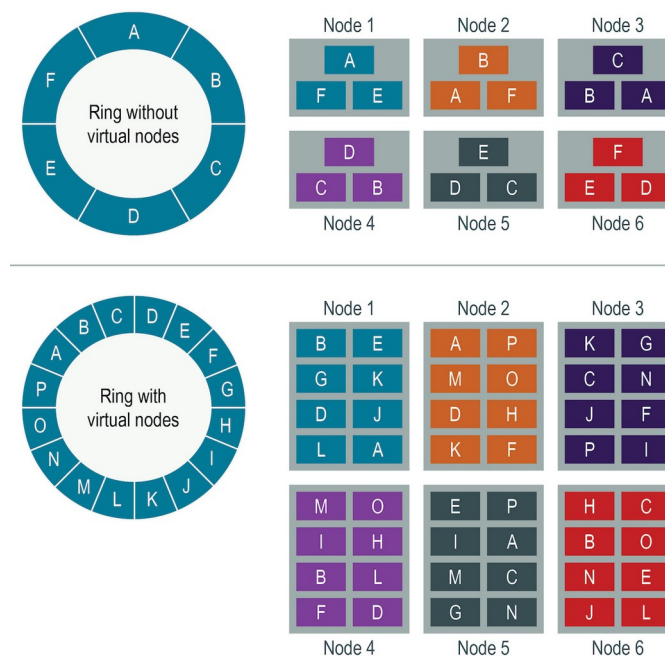


Slika 4- Consistent hashing ring kod Cassandre

### 3.2 vnodes

Konfiguracija po kojoj svaki čvor dobija po jedan token (poznata pod nazivom *simple single-token consistent hashing*) može da funkcioniše ako postoji veliki broj fizičkih čvorova. Međutim, ukoliko imamo manji broj čvorova javlja se problem prilikom dodavanja novog čvora. Radi jednostavnosti pretpostavimo da imamo heš opseg od 0 do 100 i postoji 4 čvora. Inicijalno čvorovima mogu da se dodele tokeni 0, 25, 50 i 75. Na ovaj način svaki čvor poseduje jednak deo token prstena. Kada je u ovakvoj postavci potrebno dodati novi čvor javlja se problem nebalansiranosti. Bez obzira koji se token na prstenu izabere, na primer 12, opseg novog čvora i u ovom slučaju čvora 1 biće prepolovljen.

Većina Dynamo klonova koristi *virtual nodes* (vnodes). Virtualni čvorovi rešavaju pomenuti problem tako što se svakom čvoru dodeljuje veći broj tokena na token prstenu. Dozvoljavajući jednom fizičkom čvoru da poseduje veći broj tokena na prstenu, manji clusteri izgledaju mnogo veće nego što zapravo jesu i zbog toga prilikom dodavanja samo jednog fizičkog čvora, na prstenu možemo dodati više tokena i na taj način se smanjuje procenat opsega koji se oduzima od susednih čvorova u token prstenu. [2] Na slici 5 ilustrovana je razlika između token prstena koji ne koristi vnodes i token prstena koji koristi vnodes.



Slika 5- Razlika između vnodes token ring-a i non-vnodes token ring-a

Cassandra koristi sledeću nomenklaturu kada se govori o pomenutim konceptima:

- Token – jedna pozicija na Dynamo-style heš prstenu
- Endpoint – jedna fizička IP adresa i port u mreži
- Host ID – jedinstveni identifikator jednog fizičkog čvora, obično postoji na jednom endpointu i može da ima jedan ili više tokena
- vnode – token, pri čemu veći broj vnode-ova pripada jednom čvoru (sa istim Host ID-jem)

Cassandra koristi strikturu *token map* kako bi se vodilo računa o mapiranju između tokena i endpointa. Osnovna prednost koju pristup sa većim brojem tokena po fizičkom čvoru nosi je održavanje balansiranoosti opsega, bilo u slučaju kada se doda novi fizički čvor, ili se fizički čvor

ukloni iz clustera. Pored toga, ako neki čvor postane privremeno nedostupan, opterećenje se ravnomerno raspoređuje između drugih fizičkih čvorova, naročito ako je Cassandra driver *token-aware*. [2]

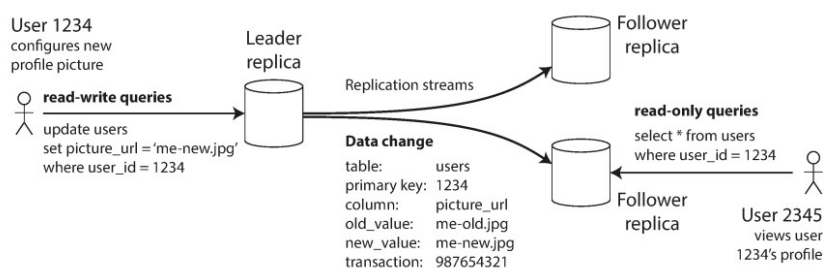
Pristup sa virtuelnim čvorovima ima i svoje nedostatke. Najveći nedostatak je uvođenje  $2 * (RF - 1)$  novih suseda u token ring-u, čime se povećava broj kombinacija otkaza čvorova gde se gubi availability dela token ringa. Kada se ne koriste virtuelni čvorovi svaki čvor ima samo jedan token (npr. A – B – C – D – E). Sa faktorom replikacije 3, opseg koji je u vlasništvu čvora A se replicira na B i na C. Kada imamo veći broj vnode-ova po fizičkom čvoru (A1 – D1 – F1 – B1 – A2 – E1 – C1 – A3...), svaki token iz A ima svoje susede gde su podaci replicirani (A sa D1 i F1, A2 sa E1 i C1, A3 na B1 i D2). Kada imamo 1 token po čvoru, otkaz čvorova A, B i C utiče na jedan range. Kada imamo više vnode-ova gubitak čvorova A, D i E može da utiče na jedan opseg, gubitak čvorova A, F i G utiče na drugi opseg itd. Pored toga, dokazano je da što je veći broj tokena, veća je verovatnoća otkaza. [2, 6] Drugi nedostatak je usporavanje maintenance operacija na clusteru, jer se povećava broj diskretnih repair operacija koje cluster mora da izvrši. Pored toga opadaju performanse operacija koje se prostiru na veći broj opsega. [2]

### 3.3 Replikacija

Replikacija je tehnika koja se koristi kako bi se kopija jednog podatka čuvala na više mašina koje su povezane, odnosno čine cluster u distribuiranom sistemu. Razlozi za replikaciju su smanjenje latencije (čuvanje podataka geografski bliže korisnicima), omogućavanje nesmetanog rada sistema i u slučaju otkaza nekih čvorova, horizontalno skaliranje operacija čitanja itd. Kompleksnost oko implementacije replikacije javlja se prilikom upravljanja promenama nad podacima. [3] U ovom poglavlju će biti pomenuti različiti tipovi replikacije, a nakon toga će detaljnije biti opisani mehanizmi koje Cassandra koristi kod replikacije.

#### 3.3.1 Single-leader replikacija

Osnovni problem koji se rešava prilikom implementacije replikacije je kako propagirati promene do svih čvorova koji drže kopiju podatka. Najčešće rešenje je leader-based replikacija. Jedan od čvorova se proglašava za leadera. Kada korisnik želi da upiše neki podatak u bazu, taj upis se šalje do leadera koji podatak upisuje kod sebe. Ostali čvorovi se nazivaju follower-i (replike). Kada leader upiše podatak kod sebe, on takođe tu promenu prosleđuje do svih replika. Svaka replika čita ove promene i upisuje ih kod sebe. Bitno je da se redosled kojim se promene upisuju u glavnom čvoru slaže sa redosledom kojim se upisuje kod replika. Prilikom čitanja podataka, klijent može da čita iz bilo kog čvora, bilo da je leader ili follower. [3] Ovakav pristup ilustrovan je na slici 6 i najzastupljeniji je kod relacionih baza podataka (Postgres, MySQL, Oracle, SQL Server), kao i kod nekih NoSql baza (MongoDB, RethinkDB).



Slika 6- Single-leader replikacija, preuzeto iz [3]

Vredi pomenuti da single-leader replikacija može biti sinhrona ili asinhrona. Kod sinhronne replikacije kada korisnik zatraži upis podatak, leader pre nego što potvrdi upis šalje promenu do replika. Korisniku se potvrđuje upis tek kada jedna ili više replika potvrde da je podatak upisan. Alternativni pristup je asinhrona replikacija, gde leader potvrđuje korisniku upis nakon što je podatak upisao kod sebe, bez čekanja odgovora od replika. Prednost sinhronne replikacije je u tome što korisnik može da bude siguran da je bar jedna replika verna kopija leader čvora. U slučaju pada leader čvora, ta replika može da bude proglašena za novog leader-a. Nedostatak je u tome što follower takođe može da otkáže i na taj način zahtev za upis ostaje da čeka dok se follower ne oporavi (potencijalno nikada). Kod asinhronne replikacije leader čvor može da nastavi da radi i u slučaju otkaza svih replika, ali se žrtvuje određen nivo konzistentnosti.

Jedan od najkomplikovanijih problema koji se sreće kod single-leader replikacije je otkaz leader-a. U tom slučaju jedan od follower-a treba da se izabere da bude novi leader. Ovaj postupak se naziva failover. Može biti automatski ili manuelni. Kod automatskog failover-a koraci koji se preduzimaju su:

1. Detektovanje da je leader otkazao – Za detekciju otkaza se najčešće koristiti heartbeat mehanizam, gde leader i replike razmenjuju heartbeat poruke u određenim intervalima.

2. Odabir novog leader-a – Ovaj problem se može rešiti korišćenjem nekog *election* algoritma, po kome replike glasaju za novog leader-a. Problem usalgašavanja čvorova u distribuiranom sistemu oko odluke je jedan od najtežih problema u distribuiranim sistemima, tzv. *consensus problem*.

3. Rekonfigurisanje sistema da se koristi novi leader – Nakon odabira novog leader-a potrebno je re-rutirati zahteve od starog do novog leader-a.

Problemi koji mogu da se jave prilikom failover-a su:

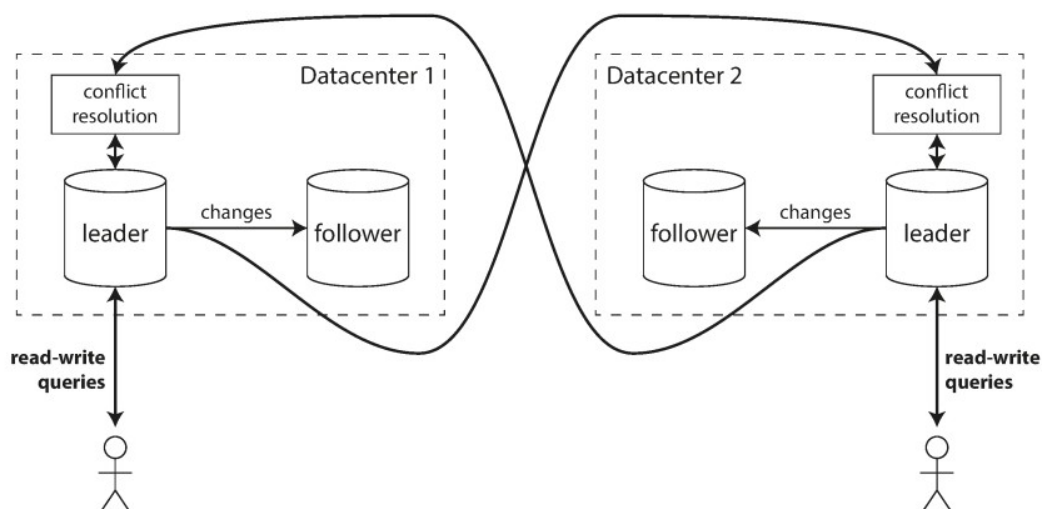
1. Ukoliko se koristi asinhrona replikacija, replike ne moraju imati najnovije podatke u trenutku kada leader otkáže. Ako se stari leader vrati u cluster nakon nekog vremena postavlja se pitanje šta raditi sa podacima koji se nalaze na njemu, naročito ako postoji konflikt između njegovih i najnovijih podataka.

2. U određenim situacijama dva čvora mogu misliti istovremeno da su leaderi, što dovodi do tzv. *split-brain* situacije.

3. Posle koliko vremena proglasiti da je leader otkazao.

### 3.3.2 Multi-leader replikacija

Nedostatak single-leader replikacije koju multi-leader replikacija pokušava da reši je upravo to postojanje samo jednog leader-a. Kod ovog pristupa dozvoljava se većem broju čvorova da prima korisničke zahteve za upis. Replikacija se i dalje dešava na isti način, svaki podataka koji neki čvor upiše mora da se propagira do svih ostalih čvorova. Korišćenje ovog pristupa ne donosi nikakve benefite u okruženju koje uključuje samo jedan data centar, ali zato ima smisla koristiti ga u okruženju koje uključuje više od jednog data centra. [3] Primer multi-leader replikacije prikazan je na slici 7. Upravljanje multi-leader replikacijom je komplikovano i dostupno je u nekim relacionim bazama podataka kroz posebne ekstenzije, kao što je Tungsten Replicator kod MySQL-a ili BDR kod Postgres-a.



Slika 7- Multi-leader replikacija

Kao što je više nego očigledno, najveći problem kod multi-leader replikacije su write konflikti koji mogu da se jave, tako da su potrebni metodi za rezoluciju konflikta.

Najjednostavnije rešenje je da se korisniku ne potvrđuje operacija sve dok se promena ne replicira do leader replike u drugom data centru. Međutim ovim pristupom se gubi najveća prednost mutli-leader replikacije, mogućnost da veći broj čvorova prima upise nezavisno.

Druga mogućnost je izbegavanje konflikata u potpunosti, tako što će zahtevi koji dolaze od istog korisnika biti rutirani do istog data centra, geografski najbližeg. Problem kod ovog pristupa je da u određenim situacijama mora da se promeni leader, odnosno „vlasnik“ reda, npr. u slučaju otkaza data centra, ili ukoliko je korisnik promenio svoju lokaciju i sada je bliži drugom data centru.

Treća mogućnost je konvergiranje ka konzistentnom stanju. Ovaj pristup podrazumeva definisanje načina za određivanje koja promena „pobeđuje“ prilikom razrešavanja konflikta. Svaki upis ima jedinstveni identifikator (uuid, timestamp, vektorski/logički časovnik). Ovakav pristup je izuzedno podložan gubitku podataka. Može se modifikovati dako da se prilikom detekcije konflikta dozvoli korisniku da odluči koji podatak treba zadržati, ali se na taj način žrtvuje određeni nivo korisničkog iskustva. [3]

Četvrta, ujedno i najinteresantnija varijanta je automatsko razrešavanje konflikata. Ovaj pristup je još uvek eksperimentalni, a nekoliko pristupa istraživanja koje se bave ovim problemom su:

- CRDT (Conflict-free replicated data types) – grupa struktura podataka koje je moguće modifikovati konkurentno sa automatskom rezolucijom konflikata. Najjednostavniji primer je *add-only set*, a neke od struktura su implementirane u Riak 2.0.
- Mergeable persistetnt data structures – strukture koje koriste three-way merge funkciju (slično kao Git)
- Operational transformation – pristup koji koristi Google Docs za kolaborativno editovanje teksta.

### 3.3.3 Leaderless replikacija

Pretpostavka na koju se oslanjaju prethodna dva pristupa replikaciji je da klijent kontaktira samo jedan čvor (leadera) kada želi da upiše podatak, a distribuirana baza je sama zadužena da podatak replicira do ostalih čvorova. Neki sistemi u potpunosti odbacuju koncept leadera i bilo koja

replika može da prihvati zahtev za upis od korisnika. Baze podataka koje koriste leaderless replikaciju su Dynamo, Riak, Cassandra, Voldemort, a baze podataka koje koriste ovaj tip replikacije nazivaju se Dynamo-style baze podataka [3].

Jedan od problema koji može da se javi kod leaderless replikacije je, kao i kod svih drugih tipova replikacije i generalno distribuiranih sistema, nedostupnost čvora. Pretpostavimo da imamo 3 čvora u clusteru i korisnik paralelno šalje zahtev za upis svim čvorovima. Dva čvora uspešno prime zahtev i potvrde upis, a treći čvor je nedostupan. Kod leaderless replikacije dovoljno je što su 2 čvora potvrdila upis. Sada pretpostavimo da je nedostupan čvor ponovo postao aktivan i sledeći zahtev za čitanje je prosleđen njemu. U tom slučaju on će vratiti zastareli podatak. Iz tog razloga se podaci kod sistema koji koriste leaderless replikaciju verzionišu, a zahtevi za čitanje se prosleđuju do više čvorova.

Pošto je većina implementacija leaderless replikacije slična, u nastavku će biti opisana leaderless replikacija u Cassandra.

### 3.4 Leaderless replikacija u Cassandra

Zvanična Cassandra dokumentacija [2] Cassandrin model replikacije naziva multi-master replikacija, što tehnički i nije netačno obzirom da je leaderless replikacija specijalan slučaj multi-leader replikacije kada su svi čvorovi leaderi.

#### 3.4.1 Strategije replikacije

Cassandra replicira svaku particiju podataka na veći broj čvorova u clusteru kako bi se omogućila visoka dostupnost i durabilnost. Kada se desi promena nad nekim podatkom, Cassandra hešira partition key kako bi se odredio opseg na token prstenu gde ključ pripada, a zatim se promena replicira do ostalih replika u zavisnosti od strategije replikacije. [2]

Sve strategije replikacije definišu *replication factor (RF)*, koji govori o tome na koliko čvorova particija treba da bude replicirana. Na primer, ako je  $RF=3$ , svaki podatak će biti upisan na tri različite replike. Replike se uvek biraju tako da se nalaze na fizički različitim čvorovima, što podrazumevanje preskakanje virtuelnih čvorova ako je to potrebno. Strategija replikacije dodatno može da dovede do preskakanje čvora u istom rack-u ili data centru te se na taj način postiže otpornost na otkaz rack-a ili data centra. Ova osobina se naziva rack-awareness.

Prva replika u koju se upisuje podatak je ona čijem opsegu na token prstenu pripada heširana vrednost ključa. Strategija replikacije nakon toga određuje preostale čvorove na kojima će se podatak replicirati. Standardna Cassandra distribucija dolazi sa dve implementacije: *SimpleStrategy* i *NetworkTopologyStrategy*. [1]

##### 3.4.1.1 SimpleStrategy

*SimpleStrategy* omogućava da se kompletno podešavanje replikacije svede na jedan ceo broj – *replication\_factor*. Ovaj broj određuje broj čvorova na kojima bi trebalo na se nalazi kopija svakog reda. *SimpleStrategy* tretira sve čvorove podjednako. Kako bi se odredile replike Cassandra se kreće po token prstenu počevši od opsega kome pripada dati ključ. Za svaki token proverava da li je vlasnički čvor već dodat u skup replika, a ukoliko nije dodaje ga. Ovaj postupak se ponavlja sve dok se ne dostigne odgovarajući broj čvorova u skupu replika. [2]

### 3.4.1.2 NetworkTopologyStrategy

NetworkTopologyStrategy zahteva specificiranje faktora replikacije za svaki data centar u clusteru. Čak i ukoliko postoji samo jedan data centar, NetworkTopologyStrategy je preporučeno rešenju u odnosu na SimpleStrategy, zbog lakšeg proširivanja clustera.

Pored toga što ova strategija omogućava specificiranje faktora replikacije po data centru, on takođe pokušava da izabere replike unutar jednog datacentra u različitim rack-ovima. U tu svrhu se koristi Snitch. Ako je broj rack-ova veći ili jednak faktoru replikacije data centra, garantovano je da će replike biti izabrane iz različitih rack-ova. Rack-aware ima i neke neočekivane osobine. Na primer, ukoliko je broj čvorova u rack-ovima nejednak, opterećenje na rack sa najmanjim brojem čvorova može da bude značajno veće. Pored toga, ako se samo jedan čvor ubaci u potpuno novi rack, on će se smatrati replikom za ceo token ring. Zbog toga često se donosi odluka da se svi čvorovi u jednom failure domenu smatraju jednim rack-om. [2]

Kao što je pomenuto, rack-awareness se postiže korišćenjem Snitch-a. Njegova uloga je da odredi relativnu blizinu između hostova. Snitch-evi prikupljaju informacije o topologiji mreže. [1] Postoji više implementacija Snitch-a:

1. Simple Snitch – podrazumevana implementacija. Ovaj snitch nije rack-aware, što ga čini praktično neupotrebljivim u multi-rack konfiguracijama.

2. PropertyFile Snitch – koristi informacije iz fajla u kome administrator opisuje topologiju mreže. Fajl se imenuje *cassandra-topology.properties*, a na slici 8 je prikazan primer kako fajl može da izgleda:

```
# Cassandra Node IP=Data Center:Rack
192.168.1.100=DC1:RAC1
192.168.2.200=DC2:RAC2
10.0.0.10=DC1:RAC1
10.0.0.11=DC1:RAC1
10.0.0.12=DC1:RAC2
10.20.114.10=DC2:RAC1
10.20.114.11=DC2:RAC1
10.21.119.13=DC3:RAC1
10.21.119.10=DC3:RAC1
10.0.0.13=DC1:RAC2
10.21.119.14=DC3:RAC2
10.20.114.15=DC2:RAC2
# default for unknown nodes
default=DC1:r1
```

Slika 8- Primer cassandra-topology.properties fajla

3. GossipingPropertyFileSnitch – čvorovi tokom gossip-a razmenjuju informacije o svojoj lokaciji, što uključuje rack i data centar. Ovaj Snitch koristi ove informacije, kao i drugi property fajl koji definiše administrator i naziva se *cassandra-rackdc.properties*.

4. RackInferring Snitch – podrazumeva da su čvorovi u mreži poređani po konstantnoj mrežnoj šemi. Ovaj Snitch jednostavno upoređuje različite oktete IP adresa svakog čvora. Ako dva čvora imaju isti drugi oktet u IP adresi, zaključuje se da pripadaju istom data centru. Ako imaju isti i treći oktet, pripadaju istom rack-u.



5. Cloud Snitches – praktično svaki Cassandra cloud provajder ima svoju implementaciju Snitch-a koji koristi karakteristike konkretnog cloud-a. Primeri cloud Snitch-eva su Ec2Snitch, Ec2MultiRegionSnitch, GoogleCloudSnitch itd.

6. Dynamic Snitch – Cassandra wrap-uje izabrani Snitch u DynamicEndpointSnitch kako bi se birali najperformantniji čvorovi za upite. Parametar *dynamic\_snitch\_badness\_threshold* definiše prag za promenu čvora. Dynamic Snitch ažurira ovaj parametar u zavisnosti od parametra *dynamic\_snitch\_update\_interval\_in\_ms* parametra.

#### **3.4.1.3 Transient replication**

Transientna replikacija je eksperimentalna funkcionalnost uvedena u verziji 4.0. Ona omogućava da se određeni podskup replika konfiguriše tako da čuva samo podatke koji još nisu inkrementalno popravljani (*incrementally repaired*). Na ovaj način se redundantnost podataka razdvaja od njihove dostupnosti.

Na primer, ako je faktor replikacije podešen na 3, a zatim se poveća na 5 dodavanjem dve transientne replike, otpornost sistema se povećava sa tolerancije otkaza jednog čvora na toleranciju otkaza dva čvora, bez proporcionalnog povećanja zauzeća skladišnog prostora. U takvoj konfiguraciji, tri čvora nastavljaju da čuvaju kompletnu kopiju podataka, dok preostala dva čvora čuvaju samo podatke koji još nisu prošli kroz inkrementalnu popravku. [2]

Tranzijentne replike trenutno podržavaju samo tabele koje su kreirane sa opcijom *read\_repair = none*. Monotona čitanja trenutno nisu podržana. Pored toga, potencijalno nikada neće moći da se koriste materijalizovani pogledi i sekundarni indeksi sa tranzijentnim replikama. Radi se o potpuno eksperimentalnoj funkcionalnosti i ne preporučuje se korišćenje u produkcionim okruženjima.

#### **3.4.1.4 Sinhronizacija replika**

Kako bilo koji čvor kod Cassandre može da primi upis ili ažuriranje podataka nezavisno od drugih čvorova, Cassandra implementira „*best-effort*“ tehnike za konvergenciju stanja replika kao što su *read repair*, *hinted handoff*, *anti-entropy repair*, *Merkle stabl*, *sub-range repair* i *incremental repair*. Više reči o ovim tehnikama biće u narednom poglavlju.

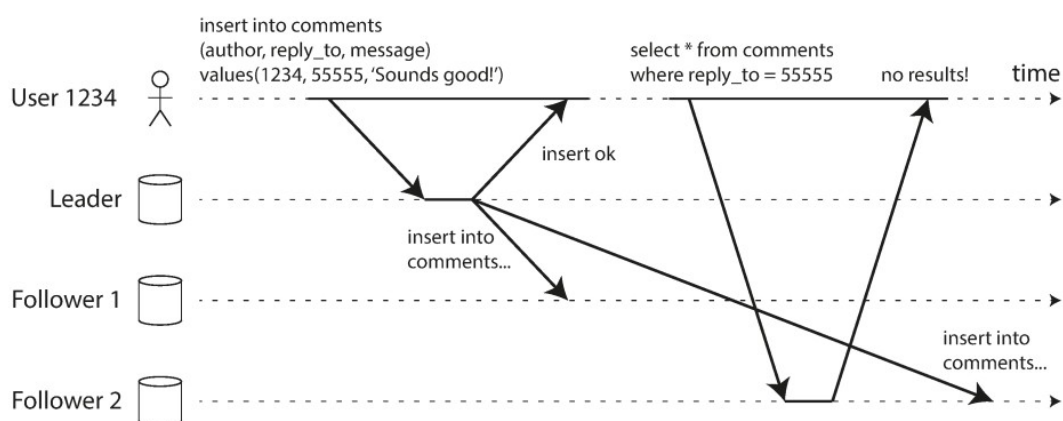
## 4 Konzistentnost kod Cassandre

### 4.1 Konzistentnost

Problem konzistentnosti kod distribuiranih sistema, samim tim i kod distribuirane baze podataka kao što je Cassandra odnosi se na situaciju kada se u dva različita čvora u istom trenutku nalaze različiti podaci. Ove nekonzistentnosti mogu da se dogode bez obzira na metodu replikacije koja se koristi (single-leader, multi-leader ili leaderless). Većina distribuiranih baza podataka obezbeđuje *eventual consistency*, što znači da od trenutka kada stane upis u bazu podataka, posle nekog nedetrimisanog vremenskog intervala, čitanja sa bilo koje replike će vraćati iste podatke. Drugim rečima, nekonzistentnost podataka je privremena i eventualno će se rešiti. [3] U nastavku će biti prikazani neki od problema koji se mogu javiti usred nesrećnog tajminga kod distribuiranih baza podataka.

#### 4.1.1 Čitaj svoje upise

Prilikom pisanja sekvencijalnih programa očekivano ponašanje je da prilikom promene vrednosti neke promenljive, sledeće čitanje iste vrati onu vrednost koja je upisana. Pored toga mnoge aplikacije dozvoljavaju korisnicima da unesu podatke, nakon čega se ti podaci prikazuju nazad. Kod asinhronne replikacije postoji problem – neposredno nakon upisa podataka, korisniku se mogu prikazati stari podaci. Na slici 9 je prikazan ovaj problem u slučaju single-leader replikacije.

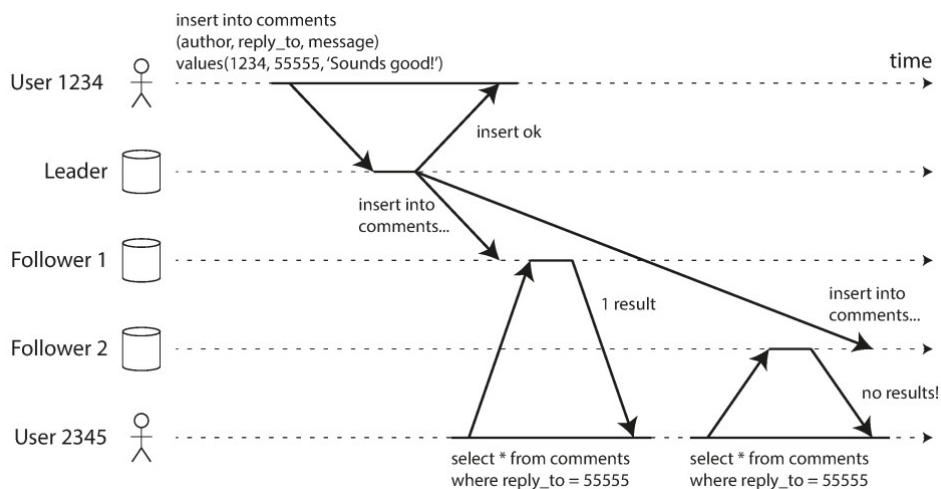


Slika 9- Ilustracija problema kada korisnik ne može da pročita podatak koji je upravo upisan

Kako bi se ovaj problem prevazišao, potrebno je obezbediti model konzistencije koji je poznat pod nazivom *read-after-write consistency*, ili čitaj svoje upise. Ovakav nivo konzistencije omogućava da korisnik koji je upisao podatke, uvek vidi najnoviju verziju tih podataka. Međutim, ovaj nivo konzistencije ne daje nikakve garancije o tome koje podatke će videti drugi korisnici, koji nisu uzrokovali promenu podatka. Implementacija ovog nivoa konzistentnosti najčešće se zasniva na korišćenju nekog timestamp-a, bilo fizičkog ili logičkog. [3]

#### 4.1.2 Monotona čitanja

Druga neželjena pojava koja može da se javi je da korisniku može da liči da se stvari kreću unazad kroz vreme. Ovakvo ponašanje može da se desi kada korisnik uputi više zahteva za čitanje do različitih replika, što je prikazano na slici 10. Korisnik 2345 je prvo čitanje izvršio sa replike (follower-a) 1, a sledeće sa replike 2, koja još uvek nije ažurirana. Na taj način korisniku 2345 liči da je korisnik 1234 zapravo obrisao komentar.

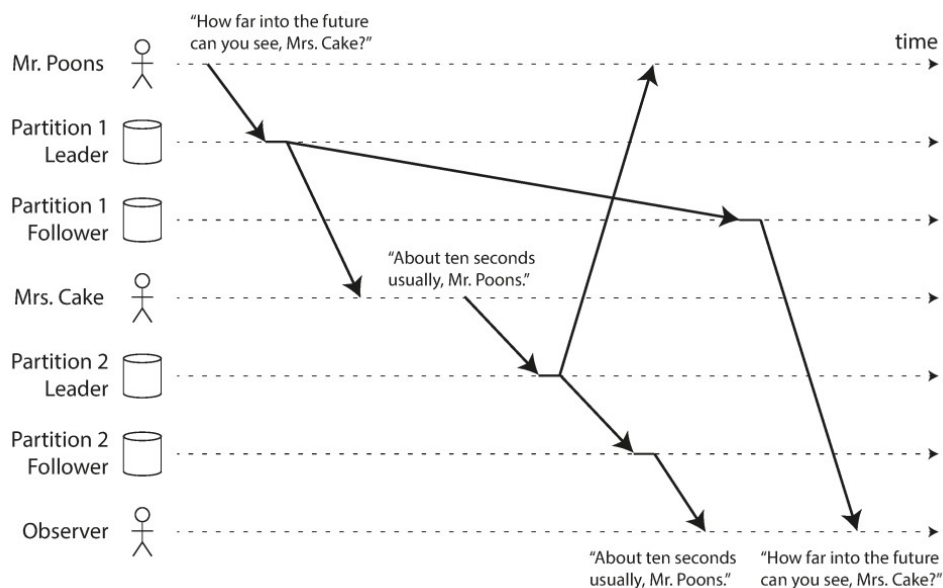


Slika 10- Primer kada se korisniku prividno stvari kreću unazad u vremenu

Za rešavanje ovog problema, potreban je stroži nivo konzistentnosti, tzv. *monotonic reads*. Ovaj nivo konzistentnosti garantuje da kada isti korisnik uputi više zahteva za čitanje, neće dobiti privid da se podaci kreću unazad. Međutim, ovaj nivo konzistencije ne garantuje da korisnik neće videti stare podatke. [3]

#### 4.1.3 Consistent prefix reads

Poslednji primer nesrećnog tajminga odnosi se na uzročnost. Primer ovog problema prikazan je na slici 11. Dva korisnika razmenjuju poruke, a treći posmatra razgovor. Zbog toga što je propagacija prve poruke nešto sporija od propagacije druge poruke, posmatrač može da se učini da je druga poruka stigla pre prve.



Slika 11- Primer narušavanja uzročnosti

Kako bi se sprečile ovakve nekonzistentnosti potreban je još stroži nivo konzistentnosti: *consistent prefix reads*. On garantuje da ako se sekvenca upisa desila u nekom redosledu, svi koji čitaju te upise videće ih u pravom redosledu. Ovakve garancije su i dalje slabije od jake konzistentnosti i naročito su komplikovane za implementaciju kod baza podataka koje koriste particije kojima se upravlja nezavisno. Jedan od pristupa koji može da se koristi su *version vectors*.

## 4.2 Kvorum

Pristup koji se koristi kod Dynamo-style baza podataka za upravljanje nivoom konzistentnosti je kvorum. Ovakav pristup se najčešće koristi kod leaderless replikacije i podrazumeva da se kontaktira veći broj čvorova i prilikom čitanja i prilikom upisa podatak u bazu. Problem koji se rešava korišćenjem kvoruma je koliko čvorova treba da prihvati upis kako bi se upis proglasio validnim, odnosno koliko čvorova treba da pročita neki podatak kako bi korisnik video najnoviju verziju podatka?

Generalno, ako postoji  $n$  replika, svaki upis mora da bude potvrđen od strane  $w$  čvorova kako bi bio uspešan i svako čitanje mora da iskoristi  $r$  čvorova. Sve dok je  $w + r > n$  može se očekivati da će svaki korisnik videti najnoviji podatak. Kod Dynamo-style baza parametri  $n$ ,  $w$  i  $r$  su obično konfigurabilni, a tipično je da se izabere da  $n$  bude neparan broj, a da  $w$  i  $r$  bude  $(n + 1) / 2$ . Na primeru clustera koji ima 3 čvora, parametri  $w$  i  $r$  su 2, što znači da se prilikom upisa i prilikom čitanja kontaktiraju 2 čvora. Ovakva konfiguracija dozvoljava jednom čvoru da bude nedostupan, a korisnik će i dalje moći nesmetano da pristupa bazi. [3]

### 4.2.1 Nivoi konzistentnosti kod Cassandre

Za Cassandru se kaže da ima *tunable consistency* jer omogućava podešavanje nivoa konzistentnosti (kvoruma) na nivou operacije. Podešavanje parametara  $w$  i  $r$  se vrši na osnovu unapred definisanih nivoa konzistentnosti bez potrebe za poznavanje faktora replikacije. Cassandra nudi sledeće nivoe konzistentnosti:

- ONE – jedna replika treba da odgovori
- TWO – dve replike moraju da odgovore
- THREE – tri replike moraju da odgovore
- QUORUM – većina ( $n / 2 * 1$ ) replika mora da odgovori
- ALL – sve replike moraju da odgovore
- LOCAL\_QUORUM – većina replika u lokalnom data centru (data centru gde se nalazi kordinatorski čvor) mora da odgovori
- EACH\_QUORUM – većina replika u svakom data centru mora da odgovori
- LOCAL\_ONE – samo jedna replika mora da odgovori. U multi-data centar konfiguracijama ovaj nivo konzistentnosti garantuje da se zahtevi za čitanje ne šalju do replika u drugim data centrima
- ANY – bilo koja replika može da odgovori, ili koordinator da skladišti hint.

Write operacije se uvek šalju do svih replika, bez obzira na nivo konzistentnosti. Nivo konzistentnosti kontroliše koliko odgovora koordinator čeka pre nego što odgovori klijentu. Kod read operacija, koordinator izdaje onoliko zahteva za čitanje da bi se zadovoljio nivo konzistentnosti, osim u slučaju da neki od čvorova ne odgovori u predviđenom intervalu.

### 4.2.2 Lightweight transakcije i Paxos

Kao što je pomenuto u prethodnom poglavlju, za Cassandru se kaže da ima *tunable consistency*, uključujući mogućnost postizanja jake konzistentnosti specificirajući dovoljno visok nivo

konzistentnosti. Međutim, jaka konzistentnost ne sprečava problem trke (*race condition*) koji može da se desi kada klijent treba da pročita, a nakon toga upiše podatak. Na primer, pre nego što kreiramo novi korisnički nalog, poželjno je proveriti da li korisnik već postoji, osim ako ne želimo da prepíšemo stari podatak. Nivo konzistentnosti koji obezbeđuje ovo naziva se *linearizable consistency*. [1]

*Linearizable consistency* nudi garanciju da ni jedan drugi klijent neće doći između operacije čitanja i upisa koji modifikuju pročitani podatak. Cassandra od verzije 2 nudi lightweight transakcije („LWT“) kojima se postiže linearizable consistency.

Cassandrina LWT implementacija se zasniva na Paxos algoritmu. Paxos algoritam je algoritam za postizanje konsenzusa u distribuiranim sistemima bez potrebe za postojanjem master čvora. Osnovni Paxos algoritam se sastoji od dve faze – prepare/promise i propose/accept. Da bi se modifikovao podatak, koordinator čvor može da predloži (propose) novu vrednost. Drugi čvorovi mogu istovremeno da budu leaderi za druge modifikacije. Svaka replika proverava predlog (proposal) i obećava da neće prihvatiti druge predloge za isti podatak. Pored toga, svaki čvor vraća poslednji predlog za taj podatak koji je i dalje u toku. Ako je predlog odobren od većine replika, leader komituje predlog, ali pre toga mora da komituje sve predloge koji su se desili pre njegovog. [1]

Cassandrina implementacija Paxos algoritma proširuje osnovni Paxos algoritam, kako bi se obezbedila read-before-write semantika. Ovo se postiže uvođenjem dve faze u Paxos algoritam, tako da su faze u Cassandrinom Paxos algoritmu:

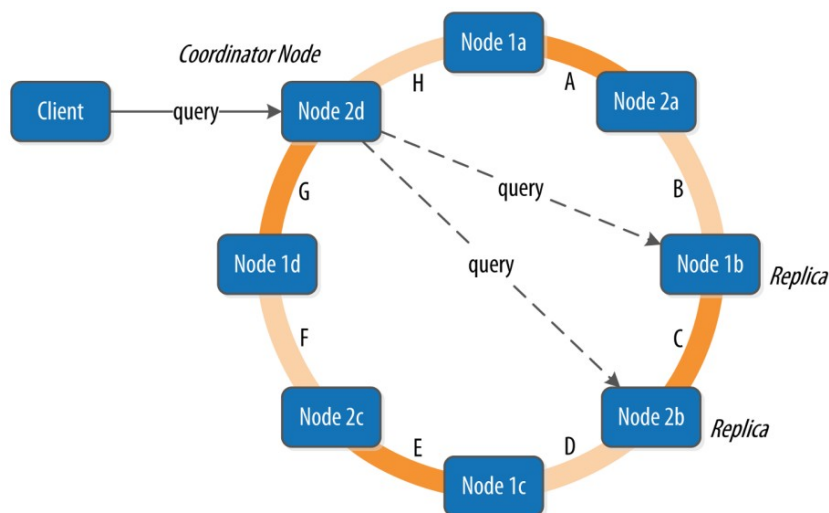
- Prepare/Promise
- Read/Results
- Propose/Accept
- Commit/Ack

Situacije u kojima se koriste LWT je potrebno pažljivo birati, obzirom da uspešna transakcija zahteva četiri obilaska od koordinatora do replika, što je skuplje od običnog upisa. [1]

## 4.3 Koordinator

Kod Cassandre klijent može da se poveže na bilo koji čvor u clusteru radi izdavanja zahteva za upis ili čitanje. Ovaj čvor se naziva koordinator. Koordinator identifikuje koji čvorovu sadrže replike podatka koji se čita ili upisuje i prosleđuje im upit.

Kod upisa, koordinator mora da iza zahtev za upis do svih replika, što je određeno faktorom replikacije, a upis se smatra uspešnim kada broj čvorova koji odgovara nivoom konzistentnosti vrati odgovor. Kod čitanja, koordinator kontaktira dovoljno replika kako bi se zadovoljio odgovarajući nivo konzistentnosti, a nakon čega se klijentu vraća odgovor. [1] Na slici 12 prikayana je ilustracija ovog postupka, a sam postupak se odnosi na „*happy path*“, odnosno kada su sve replike dostupne i odgovore na vreme.



Slika 12- Koordinator čvor kod Cassandre, preuzeto iz [1]

#### 4.4 Sloppy kvorum i hinted handoff

Kvorum koji je opisan u prethodnim poglavljima nije fault-tolerant koliko bi mogao da bude. Problem u mreži može da odseče klijenta od velikog broja čvorova koji čine bazu. Iako su ti čvorovi živi, za klijenta koji ima mrežne probleme, oni mogu da se posmatraju i kao mrtvi. U takvom slučaju velika je verovatnoća da klijent neće moći da postigne kvorum koji bi zadovoljio parametre  $r$  i  $w$ . Prilikom dizajna arhitekture distribuiranog sistema u ovakvom slučaju postoje dve mogućnosti:

- Klijentu prijaviti grešku za sve zahteve koji ne mogu da uspostave kvorum
- Prihvatiti upis, upisati u neki čvor koji je dostupan, ali nije među  $n$  čvora gde se podatak obično nalazi

Druga varijanta poznata je pod nazivom *sloppy quorum*. Upisi i čitanja i dalje zahtevaju kvorum od  $w$  i  $r$  čvorova, ali oni mogu da uključe i čvorove koji nisu među  $n$  čvorova gde je podatak repliciran. Kada se problem sa mrežom reši, bilo koji upis u privremene čvorove će biti prosleđen do čvorova koji su trebali da čine originalni kvorum. Ovaj postupak se naziva *hinted handoff*. [3]

Sloppy kvorumi su izuzetno korisni za povećavanje dostupnosti kod upisa, jer dok god postoji  $w$  dostupnih čvorova upis će biti uspešan. Međutim, korišćenjem sloppy kvoruma žrtvuje se određen nivo konzistentnosti kod čitanja, obzirom da je podatak možda upisan u privremeni čvor. Sloppy kvorumi su implementirani kod svih Dynamo-style baza podataka. Kod Cassandre su po default-u isključeni.

Jedan potencijalni problem koji može da se javi kod *hinted handoff* procesa je da ukoliko je čvor koji treba da primi upise nedostupan duže vreme, *hintovi* mogu da se nagomilaju u ostalim čvorovima. Nakon toga, kada čvorovi koji čuvaju hintove detektuju da je čvor ponovo dostupan oni mogu preplaviti čvor zahtevima za upis. Kako bi se prevazišao ovaj problem, Cassandra ograničava broj hintova koje čvor može da skladišti u određenom vremenskom periodu. [1]

## 4.5 Repair

Cassandra je projektovana tako da ostane dostupna i ako je jedan od čvorova nedostupan ili je pao. Međutim, kada čvor ponovo postane dostupan potrebno je da eventualno nadoknadi sve upise koje je propustio. Hinted handoff je jedan best effort pristup, a Cassandra ne garantuje da će čvor videti sve propuštene upise. Ovakva nekonzistentnost može dovesti do gubitka podataka. U tu svrhu Cassandra nudi *repair* proces. *Repair* sinhronizuje podatke između čvorova tako što upoređuje podatke koje čvorovi imaju u zajedničkim opsezima na token prstenu i strimuje razlike do čvora koji je nesinhronizovan. Za implementaciju *repair-a* Cassandra koristi merkle tree strukturu podataka. [2]

Cassandra implementira dva tipa *repair-a*: *full repair* i *incremental repair*. Full repair upoređuje sve podatke u opsegu na token prstenu koji se sinhronizuje. Sa druge strane incremental repair upoređuje samo podatke koji su bili upisani od poslednjeg inkrementalnog *repair-a*. Incremental repair su podrazumevani tip *repair-a* i ukoliko se izvršavaju redovno mogu značajno da smanje cenu IO operacija prilikom full *repair-a*. Jednom kada se neki podatak označi kao „popravljen“ korišćenjem incremental *repair-a* on se više neće sinhronizovati od strane narednih incremental *repair-ova*. Zbog toga incremental repair ne štiti od gubitka podataka usled problema sa diskom, gubitka izazvanog od strane operatora ili bugova u Cassandri. Zbog toga se preporučuje da se full repair periodično izvršava. [2]

## 4.6 Read repair

*Read repair* je proces „popravljanja“ podataka tokom zahteva za čitanje. Ako su sve replike koje učestvuju u procesu čitanja, u skladu sa nivoom konzistentnosti, konzistentne ne vrši se nikakav *repair* proces. Međutim, ukoliko podaci koji se prikupe od replika koje učestvuju u operaciji čitanja nisu konzistentni pokreće se *read repair* proces kojim se sve replike sinhronizuju, a korisniku se vraća najnoviji podatak. Read repair se izvršava sinhrono, odnosno blikira izvršenje zahteve dok se *repair* proces ne završi. [2]

Read repair je mehanizam koji Cassandra koristi kako bi obezbedio *monotonic read* nivo konzistentnosti, odnosno kako bi se kod dva uzastupna zahteva za čitanje, garantovalo da se kod drugog neće vratiti nešto starije nego kod prvog.

Od verzije 4, Cassandra nudi mogućnost podešavanja *read repair* procesa na nivou tabele, korišćenjem parametra *read\_repair*. Ovaj parametar može da ima dve vrednosti – *blocking* i *none*. Vrednost *blocking* obezbeđuje *monotonic read* nivo konzistentnosti. None vrednost parametra garantuje *write atomicity*. Write atomicity sprečava *read* operacije da vrate parcijalno primenjene *write* operacije. Cassandra pokušava da obezbedi *write atomicity* na nivou particije, ali pošto se *repair* vrši samo nad podacima koji se nalaze u SELECT delu upita, *read repair* može da dovede do nekonzistentnosti na nivou reda kada se podaci čitaju sa većom granularnošću nego što se upisuju. [2]

## 4.7 Read (repair) path

U poglavlju 4.3 opisan je postupak kako se *read* operacije odvijaju kod Cassandre, korišćenjem čvora koji je koordinatorski. U ovom poglavlju će *read path* biti opisan sa više detalja, uključujući i *read repair* proces.

Pretpostavimo da je prilikom operacije čitanja nivo konzistentnosti postavljen na TWO, u clusteru sa 5 čvorova, i neka je faktor replikacije tri. Obzirom da je nivo konzistentnosti postavljen na dva, to znači da podatak treba da se pročitati sa dve replike pre nego što se vrati klijentu. Ako je čvor do koga je stigao zahtev (koordinator) ujedno i čvor na kome se nalazi jedna od replika, u tom slučaju se zahtev šalje samo do još jedne replike. Ukoliko koordinator ne hostuje repliku, u tom slučaju je potrebno poslati zahtev do dva čvorova u clusteru. [2]

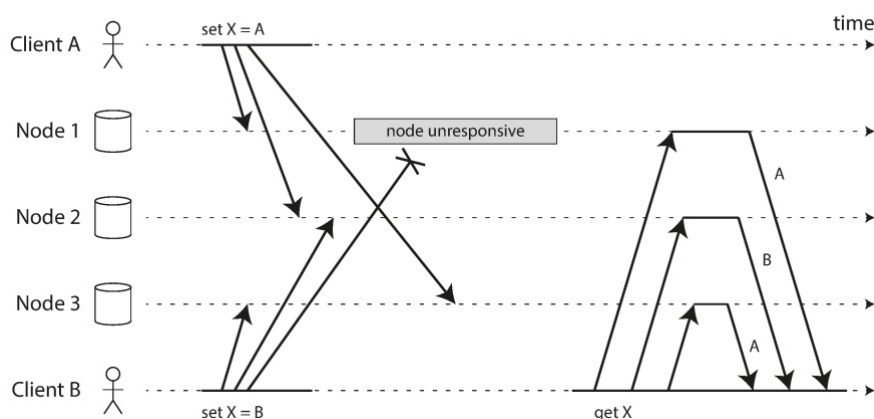
U slučaju da koordinator ne hostuje repliku, prvi zahtev koji koordinator čvor šalje je tzv. *direct read request* do čvora koji je najbrži. Ova informacija se dobija od odgovarajućeg *snitch-a*. Nakon toga koordinator šalje preostale zahteve kako bi se ispunio kvorum, odnosno odgovarajući nivo konzistencije. Svi naredni zahtevi su *digest read request*, koji ne vraćaju ceo podatak, već samo heš vrednost podatka, kako bi se smanjio mrežni saobraćaj. Sada koordinator kod sebe ima jednu potpunu vrednost podatka i jedan heš, nakon čega upoređuje heš potpune verzije sa hešom koji je prikupljen. Ukoliko se heševi ne razlikuju, ne pokreće se read repair proces, a korisniku se vraća kompletna verzija podatka. Međutim, ukoliko se heševi razlikuju, pokreće se read repair proces. [2]

Read repair process počinje tako što koordinator šalje direct read request do čvorova kojima je poslat digest read request. Koordinator upoređuje podatke iz dva izvora i određuje koja je novija verzije (ili radi merge ukoliko su neke kolone iz prvog izvora novije, a neke iz drugog). Nakon toga koordinator šalje read repair zahteve do svih čvorova kod kojih treba popraviti podatke. [2]

Cassandra je u verziji 4 uvela dva unapređenja u read repair proces. Prvi je spekulativni retry full data read requesta. Kada Cassandra proceni da neki od čvorova koji su inicijalno izabrani da budu deo kvoruma možda neće odgovoriti, spekulativno se šalju dodatni zahtevi prethodno nekontaktiranim čvorovima. Cassandra će takođe, spekulativno poslati repair zahteve i do čvorova koji nisu učestvovali u read operaciji. Drugo poboljšanje je blokirajuća priroda read repair-a, odnosno Cassandra će blokirati zahtev samo dok se ne odredi koja verzija podatka je najnovija.

## 4.8 Konkurentni upisi

Dynamo-style baze podataka dozvoljavaju većem broju klijenta da istovremeno menjaju (upisuju/ažuriraju) podatak sa istim ključem, što znači da će se konflikti desiti, osim ako se ne koristi LWT. Primer problema koji može da nastupi prikazan je na slici 13. Čvor 1 ne dobije zahtev za upis od korisnika B zbog problema u mreži, čvor 2 dobije upis od korisnika A pre upisa od korisnika B, a čvor tri dobije upis od korisnika B pre nego što dobije upis od korisnika A. [3]



Slika 13- Problem kod konkurentnih upisa kod Dynamo-style baza podataka



Za rešavanje ovog problema Cassandra koristi *last write wins (LWW)* pristup. Ideja je da se uz uvede poredak među operacije i da se čvorovima dozvoli da pamte samo najnoviju verziju, a da se starije verzije prepisu. Sa slike 13, jasno se vidi da operacije nemaju prirodni poredak, te se zbog toga i zovu konkurentne operacije. Moguće je, međutim, na silu uvesti poredak, tako što će se svakoj operaciji pridružiti timestamp. Kada dođe do konflikta bira se noviji podatak i odbacuje stari. Ovo je jedini metod za rezoluciju konflikata kod Cassandre. [3] Cassandra za timestamp koristi klijentsko vreme na koordinatorskom čvoru te je sinhronizacija vremena korišćenjem NTP protokola preporučljiva kod mašina na kojima se izvršava Cassandra. [2] Ipak, najbolje rešenje je izbegavanje konkurentnih upisa tako što će se podatak upisivati samo jednom i to korišćenjem uuid-a kao ključ.

## 5 Praktični prikaz

### 5.1 Cassandra cluster sa tri čvora

U okviru ovog praktičnog prikaza pokreće se Cassandra cluster sa tri čvora. Docker-compose konfiguracija jednog čvora je prikazana na slici 14, a identična je i za ostale čvorove.

```
services:
  cassandra-node1:
    image: cassandra:5.0.8
    container_name: cassandra-node1
    hostname: cassandra-node1
    ports:
      - "9042:9042"
    environment:
      - CASSANDRA_CLUSTER_NAME=MyCluster
      - CASSANDRA_SEEDS=cassandra-node1
      - CASSANDRA_DC=datacenter1
      - CASSANDRA_RACK=rack1
      - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
      - MAX_HEAP_SIZE=512M
      - HEAP_NEWSIZE=100M
    deploy:
      resources:
        limits:
          memory: 1024M
    volumes:
      - cassandra-data1:/var/lib/cassandra
    networks:
      - cassandra-net
    healthcheck:
      test: ["CMD-SHELL", "cqlsh -e 'SELECT release_version FROM system.local'"]
      interval: 15s
      timeout: 10s
      retries: 10
      start_period: 60s
```

Slika 14- Docker-compose konfiguracija jednog čvora u clusteru

### 5.2 Faktor replikacije 1

Najpre će biti kreiran keyspace koji ima faktor replikacije 1, što praktično znači da se podaci ne repliciraju na više čvorova. Primeri kreiranja takvog keyspace-a i testne tabele prikazani su na slikama 15 i 16, a insert par redova na slici 17.

```
cqlsh> CREATE KEYSPACE keyspace_rf1
... WITH replication = {
...   'class': 'NetworkTopologyStrategy',
...   'datacenter1': 1
... };
```

Slika 16- Kreiranje keyspace-a sa rf=1

```
cqlsh:keyspace_rf1> CREATE TABLE users (
...   user_id uuid,
...   first_name text,
...   last_name text,
...   email text,
...   created_at timestamp,
...   PRIMARY KEY (user_id)
... );
```

Slika 15- Kreiranje tabele unutar keyspace-a sa rf 1

```
cqlsh:keyspace_rf1> INSERT INTO users (user_id, first_name, last_name, email, created_at)
... VALUES (uuid(), 'Luka', 'Velickovic', 'luleee@elfak.rs', toTimestamp(now()));
cqlsh:keyspace_rf1> INSERT INTO users (user_id, first_name, last_name, email, created_at)
... VALUES (uuid(), 'Pera', 'Peric', 'pera.peric@gmail.com', toTimestamp(now()));
```

Slika 17- Insert nekoliko redova u testnu tabelu

Cassandra nudi nodetool alat koji ima puno korisnih funkcionalnosti, među kojima je i prikaz svih čvorova koji sadrže repliku nekog podatka na osnovu primarnog ključa. Prikaz čvorova koji sadrže upravo insertovane podatke prikazan je na slici 18.

```
lule@lule-Asus-Vivobook:~$ sudo docker exec -it cassandra-node1 nodetool getendpoints keyspace_rf1 users "e364bcb8-a05d-4e03-aa45-eb30d656d741"
172.18.0.2
lule@lule-Asus-Vivobook:~$ sudo docker exec -it cassandra-node1 nodetool getendpoints keyspace_rf1 users "07b2a302-a539-485b-bd13-62177d37448c"
172.18.0.4
```

Slika 18- Prikaz ip adrese čvorova koji sadrže repliku podatka

Kao što se može videti, oba podatka se nalaze na jednom čvoru. Sada će čvor koji sadrži jedan od podataka biti isključen, a zatim će biti izvršen upit za pribavljanje svih korisnika u bazi.

```
cqlsh> select * from keyspace_rf1.users allow filtering;
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level ONE" info={\'consistency\': \'ONE\', \'required_replicas\': 1, \'alive_replicas\': 0}\')})
```

Slika 19- Izvršenje upita nakon što je čvor koji sadrži podatak ugašen

Kao što se može videti, Cassandra neće dozvoliti izvršavanje upita ukoliko ne može da se uspostavi odgovarajući nivo konzistentnosti za pribavljanje podatka. Obzirom da je jedini čvor koji sadrži podatak nedostupan, Cassandra ne može da ispuni ni najslabiji nivo konzistentnosti ONE, te upit rezultuje greškom, koja je prikazana na slici 19. Keyspace sa faktorom replikacije 1 je iskorišćen za demonstraciju ove osobine zbog jednostavnosti. Ovakvo ponašanje se izražava bez obzira na broj data centara, rackova i faktor replikacije, ukoliko Cassandra ne može da uspostavi kvorum. Opisano ponašanje je očekivano i prilikom write operacija, čak i ako je hinted handoff uključen!

```
cqlsh> INSERT INTO keyspace_rf1.users (user_id, first_name, last_name, email)
... VALUES (3f76a788-ea30-48e0-8271-dc175b30219c, 'Hinted', 'Handoff', 'hinde
id@handoff.com');
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level ONE" info={\'consistency\': \'ONE\', \'required_replicas\': 1, \'alive_replicas\': 0}\')})
```

Slika 20- Insert operacije, kada je čvor koji je vlasnik tog reda nedostupan

Korišćenje faktora replikacije 1 kod distribuirane baze podataka nema smisla, ali je u ovom prikazu iskorišćen zbog jednostavnije postavke za demonstraciju ponašanja u specifičnim situacijama.

## 5.3 Faktor replikacije 2

U narednim prikazima koristiće se keyspace sa faktorom replikacije 2, kao i tabela prikazana na slici 15.

```
cqlsh> CREATE KEYSPACE keyspace_rf2
... WITH replication = {
...     'class': 'NetworkTopologyStrategy',
...     'datacenter1': 2
... };

```

Slika 21- Kreiranje keyspace-a sa faktorom replikacije 2

Najpre će biti izvršene iste insert operacije koje su prikazane na slici 17, nakon čega će ponovo biti iskorišćen nodetools kako bi se prikazali čvorovi koji sadrže replike upravo insertovanih podataka. Prikaz izvršenja ovih komandi prikazan je na slici 22.

```
lule@lule-Asus-Vivobook:~$ sudo docker exec -it cassandra-node1 nodetool getendpoints keyspace_rf2 users "fff5fa85-07d3-4871-a10d-824fabe364c4"
172.18.0.4
172.18.0.3
lule@lule-Asus-Vivobook:~$ sudo docker exec -it cassandra-node1 nodetool getendpoints keyspace_rf2 users "51e803e6-3eca-4a30-bfcc-235922ee4775"
172.18.0.3
172.18.0.2
```

Slika 22- Prikaz čvorova koji sadrže replike podataka

### 5.3.1 Hinted handoff

Ova sekcija odnosi se na demonstraciju hinted handoff-a. Kako bi se demonstrirao hinted handoff, najpre je potrebno pronaći primarni ključ koji se replicira na određenom čvoru, a nakon toga se ugasi taj čvor iza čega se izvrši upis podataka. Simuliraće se otkaz čvora 3, tako da je najpre potrebno pronaći jedan od opsega na token prstenu koji pripada čvoru 3. Primer kako se to može uraditi nalazi se na slici 23.

```
lule@lule-Asus-Vivobook:~$ sudo docker exec -it cassandra-node1 nodetool ring

Datacenter: datacenter1
=====
Address            Rack    Status State  Load             Owns               Token
172.18.0.4          rack1   Up      Normal 168.25 KiB        ?                 9008140603904522259
172.18.0.2          rack1   Up      Normal 249.03 KiB        ?                 -9078584325173739591
172.18.0.4          rack1   Up      Normal 168.25 KiB        ?                 -8798163910563576400
172.18.0.3          rack1   Up      Normal 182.24 KiB        ?                 -8416452818871498354
172.18.0.3          rack1   Up      Normal 182.24 KiB        ?                 -8180182878661376322
172.18.0.2          rack1   Up      Normal 249.03 KiB        ?                 -7726663294223543587
172.18.0.4          rack1   Up      Normal 168.25 KiB        ?                 -7354100419201146814
172.18.0.3          rack1   Up      Normal 182.24 KiB        ?                 -7115841888888312925
```

Slika 23- Prikaz nekoliko token opsega

Ip adresa čvora 3 je 172.18.0.4, što znači da je jedan od opsega koji pripadaju čvoru 3 (-8798163910563576400, -8416452818871498354]. Zapravo, čvoru tri pripada i opseg (9008140603904522259, -9078584325173739591], koji je prvi opseg prikazan, ali zbog jednostavnijeg nalaženja potrebnog uuid-a koristiće se drugi opseg koji mu pripada. Za pronalaženje odgovarajućeg uuid-a koristiće se brute force pristup prikazan na slici 24.

```
import uuid
import mmh3

MIN_TOKEN = -8798163910563576400
MAX_TOKEN = -8416452818871498354

while True:
    u = uuid.uuid4()
    token = mmh3.hash64(u.bytes, seed=0, signed=True)[0]

    if MIN_TOKEN <= token <= MAX_TOKEN:
        print(f"Target UUID: {u}")
        break
```

Slika 24- Brute force pristup za pronalaženje uuid koji pripada opsegu nekog čvora na osnovu granica opsega

Sada kada imamo uuid možemo da simuliramo nedostupnost čvora gašenjem, a zadim možemo da probamo da izvršimo insert podatka.

```
cqlsh> INSERT INTO keyspace_rf2.users (user_id, first_name, last_name, email)
... VALUES (8f7fc9b6-3914-4064-bcc5-6c580eca2bd5, 'Hinted', 'Handoff', 'hinded@handoff.com');
cqlsh>
```

Slika 25- Uspešan insert podatka, iako je čvor koji je vlasnik reda nedostupan

Nakon ponovnog pokretanja prethodno ugašenog čvora, nakon par sekundi može se primetiti log na jednom od preostalih čvorova o uspešno završenom hinted handoff-u, što je i prikazano na slici 26.

```
INFO [HintsDispatcher:1] 2026-06-08T19:18:02,809 HintsStore.java:232 - Deleted hint file b2133fc2-eb10-4d62-8a7a-14abae2f00d5-1780946116664-2.hints
INFO [HintsDispatcher:1] 2026-06-08T19:18:02,809 HintsDispatchExecutor.java:300 - Finished hinted handoff of file b2133fc2-eb10-4d62-8a7a-14abae2f00d5-1780946116664-2.hints to endpoint /172.18.0.4:7000: b2133fc2-eb10-4d62-8a7a-14abae2f00d5
```

Slika 26- Log o uspešno završenom hinted handoff-u

## 5.4 Nivoi konzistencije

U ovoj sekciji biće korišćen isti keyspace (sa faktorom replikacije 2) i tabela prikazani na slikama 21 i 15. Ideja je upoređivanje performansi čitanja u zavisnosti od nivoa konzistencije koji je podešen za operaciju. Pošto je broj čvorova u clusteru 3, a faktor replikacije je 2, nivoi koji će biti ispitani su ONE, QUORUM i ALL, odnosno čitanje koje zahteva da se podataka vrati sa jednog, dva ili sa sva tri čvora.

Pre svega, u Cassandra je upisano 100\_000 redova korišćenjem skripte prikazane na slici 27.

```
import time
from cassandra.cluster import Cluster
from cassandra.concurrent import execute_concurrent_with_args
from faker import Faker

fake = Faker()

print("Connecting to Cassandra cluster...")
cluster = Cluster(['127.0.0.1'], port=9042)
session = cluster.connect('keyspace_rf2')

insert_stmt = session.prepare("""
    INSERT INTO users (user_id, first_name, last_name, email, created_at)
    VALUES (uuid(), ?, ?, ?, toTimestamp(now()))
""")

TOTAL_ROWS = 100_000
BATCH_SIZE = 2_000

print(f"Starting bulk insert of {TOTAL_ROWS:,} rows...")
start_time = time.time()

inserted_count = 0
while inserted_count < TOTAL_ROWS:
    data_batch = []
    for _ in range(BATCH_SIZE):
        first_name = fake.first_name()
        last_name = fake.last_name()
        email = f"{first_name.lower()}.{last_name.lower()}@{fake.free_email_domain()}"
        data_batch.append((first_name, last_name, email))

    results = execute_concurrent_with_args(session, insert_stmt, data_batch, concurrency=100)

    inserted_count += BATCH_SIZE
    elapsed = time.time() - start_time
    print(f"Progress: {inserted_count:,}/{TOTAL_ROWS:,} rows inserted... ({elapsed:.2f}s elapsed)")

cluster.shutdown()

end_time = time.time()
print(f"Total time taken: {end_time - start_time:.2f} seconds")
print(f"Average speed: {int(TOTAL_ROWS / (end_time - start_time)):,} rows/sec")
```

Slika 27- Skripta kojom je upisano 100\_000 redova u Cassandra

Postupak testiranja vremena izvršenja je takođe automatizovan korišćenjem skripte prikazane na slici 28.

```

import random
import time
from cassandra import ConsistencyLevel
from cassandra.cluster import Cluster

cluster = Cluster(['127.0.0.1'], port=9042)
session = cluster.connect('keyspace_rf2')

sample_query = "SELECT user_id FROM users LIMIT 3000"
rows = session.execute(sample_query)
all_user_ids = [row.user_id for row in rows]
random.shuffle(all_user_ids)

test_data = {
    "ALL": all_user_ids[0:1000],
    "ONE": all_user_ids[1000:2000],
    "QUORUM": all_user_ids[2000:3000]
}
consistency_levels = {
    "ONE": ConsistencyLevel.ONE,
    "QUORUM": ConsistencyLevel.QUORUM,
    "ALL": ConsistencyLevel.ALL
}

for name, level in consistency_levels.items():
    print(f"Testing consistency level: {name}...", end="", flush=True)

    query_str = "SELECT * FROM users WHERE user_id = ?"
    statement = session.prepare(query_str)
    statement.consistency_level = level

    current_ids = test_data[name]

    start_time = time.time()

    for uid in current_ids:
        session.execute(statement, (uid,))

    end_time = time.time()

    total_time = end_time - start_time
    avg_latency = (total_time / len(current_ids)) * 1000 # u milisekundama

    print(f"\rLevel: {name:<8} | Total time: {total_time:.4f} s | Average latency: {avg_latency:.2f} ms")

print("-" * 75)

cluster.shutdown()

```

Slika 28- Skripta za testiranje performansi nivoa konzistentnosti

Skripta se pokreće više puta, pri čemu se između izvršenja menja broj upita koji se izvršava. Pored toga, inicijalno se randomizuju ključevi koji će se pretraživati u okviru nivoa konzistencije, kako bi se ublažilo dejstvo keša. Rezultati eksperimenta prikazani su u tabeli 1.

| Nivo konzistencije | 100            |             | 1000           |             | 10_000         |             |
|--------------------|----------------|-------------|----------------|-------------|----------------|-------------|
|                    | Total time [s] | Latency [s] | Total time [s] | Latency [s] | Total time [s] | Latency [s] |
| ONE                | 0.05           | 0.55        | 0.58           | 0.58        | 4.1            | 0.41        |
| QUORUM             | 0.1            | 1.03        | 1.1            | 1.1         | 8.2            | 0.82        |
| ALL                | 0.09           | 0.91        | 0.79           | 0.8         | 8.95           | 0.9         |

Pri najvećem uzorku (10\_000 redova), keširanje i statički šum gube uticaj i sistem kreće da se ponaša prilično približno teoriji. Nivo konzistencije ONE je očekivano najbrži, obzirom da koordinator šalje zahtev samo jednoj replici (potencijalno i ne šalje, ukoliko on sadrži podatak). QUORUM nivo konzistentnosti zahteva odgovor od 2 čvora, što su dva round trip zahteva, pa je i vreme duplo veće nego nivo konzistencije ONE. Poslednji nivo konzistentnosti – ALL je nešto sporiji od QUORUMA, možda manje spor nego očekivano, ali tu u obzir ulazi nekoliko faktora –

kod ALL nivoa konzistencije zahtevi mogu odmah da se pošalju do svih čvorova, a kod QUORUM-a je potrebno obilaziti token ring sve dok se ne utvrde replike koje treba kontaktirati. Pored toga QUORUM u nezanemarljivom broju situacija šalje isti broj zahteva kao i ALL nivo konzistencije. Slično se može objasniti i razlog zbog čega je QUORUM nivo konzistencije nešto sporiji od ALL nivoa kada se uzorak manji.

## 5.5 Lightweight transakcije

U okviru ove sekcije biće upoređene performanse standardnih upisa i lightweight transakcija. Za testiranje iskorišćena je skripta prikazana na slici 29. Rezultat izvršenja skripte prikazan je na slici 30.

```
import time
import uuid
from cassandra import ConsistencyLevel
from cassandra.cluster import Cluster

cluster = Cluster(['127.0.0.1'], port=9042)
session = cluster.connect('keyspace_rf2')

NUM_OPERATIONS = 1000

regular_stmt = session.prepare("""
    INSERT INTO users (user_id, first_name, last_name, email, created_at)
    VALUES (?, 'Regular', 'User', 'regular@example.com', toTimestamp(now()))
""")
regular_stmt.consistency_level = ConsistencyLevel.QUORUM
regular_uuids = [uuid.uuid4() for _ in range(NUM_OPERATIONS)]

start_time = time.time()
for uid in regular_uuids:
    session.execute(regular_stmt, (uid,))
end_time = time.time()

reg_total_time = end_time - start_time
reg_avg_latency = (reg_total_time / NUM_OPERATIONS) * 1000

print(f"\r[REGULAR] Total time: {reg_total_time:.4f} s | Avg latency: {reg_avg_latency:.2f} ms")

lwt_stmt = session.prepare("""
    INSERT INTO users (user_id, first_name, last_name, email, created_at)
    VALUES (?, 'LWT', 'User', 'lwt@example.com', toTimestamp(now()))
    IF NOT EXISTS
""")
lwt_stmt.consistency_level = ConsistencyLevel.QUORUM
lwt_uuids = [uuid.uuid4() for _ in range(NUM_OPERATIONS)]

start_time = time.time()
for uid in lwt_uuids:
    session.execute(lwt_stmt, (uid,))
end_time = time.time()

lwt_total_time = end_time - start_time
lwt_avg_latency = (lwt_total_time / NUM_OPERATIONS) * 1000

print(f"\r[LWT    ] Total time: {lwt_total_time:.4f} s | Avg latency: {lwt_avg_latency:.2f} ms")
cluster.shutdown()
```

Slika 29- Skripta za upoređivanje performansi LWT-a

```
[REGULAR] Total time: 0.5819 s | Avg latency: 0.58 ms
[LWT    ] Total time: 2.6901 s | Avg latency: 2.69 ms
```

Slika 30- Rezultat izvršenja skripte sa slike 29

U ovom slučaju dobijeni su očekivani rezultati. Lightweight transakcije su značajno sporije od regularnih upisa zbog mnogo većeg mrežnog saobraćaja koji se odvija prilikom izvršenja Cassandrine proširene implementacije Paxos algoritma.



## 5.6 Anty-entropy repair

Poslednja sekcija u poglavlju posvećenom praktičnom prikazu je repair operacije. Za demonstraciju koristi se isti keyspace i tabela kao u prethodnoj sekciji, a prikazani su na slikama 21 i 15. Najpre će se jedan čvor ugasiti kako bi se simulirala nedostupnost čvora i biće ugašen hinted handoff na ostalim čvorovima kako se replike ne bi same sinhronizovale prilikom ponovnog pokretanja ugašenog čvora. Postupak isključivanja hinted handoff-a prikazan je na slici 31.

```
lule@lule-Asus-Vivobook:~/Desktop/DBMS/P2/impl$ sudo docker exec -it cassandra-node1 nodetool disablehandoff
lule@lule-Asus-Vivobook:~/Desktop/DBMS/P2/impl$ sudo docker exec -it cassandra-node2 nodetool disablehandoff
```

Slika 31- Isključivanje hinted handoff mehanizma

Nakon toga, u Cassandra je upisano par hiljada podataka korišćenjem skripte sa slike 27, a zatim je ponovo uključen prethodno ugašeni čvor. Primer uspešno izvršenog repair-a prikazan je na slici 32.

```
lule@lule-Asus-Vivobook:~/Desktop/DBMS/P2/impl$ sudo docker exec -it cassandra-node1 nodetool
repair keyspace_rf2 users
[2026-06-08 20:31:32,729] Starting repair command #1 (09865ff0-6379-11f1-8c9d-2ddff7a14c59),
repairing keyspace keyspace_rf2 with repair options (parallelism: parallel, primary range: fa
lse, incremental: true, job threads: 1, ColumnFamilies: [users], dataCenters: [], hosts: [],
previewKind: NONE, # of ranges: 32, pull repair: false, force repair: false, optimise streams
: false, ignore unreplicated keyspaces: false, repairPaxos: true, paxosOnly: false)
[2026-06-08 20:31:32,970] Repair session 099e06a0-6379-11f1-8c9d-2ddff7a14c59 for range [(519
199489102755587,800793225788335170], (6634366274940308527,6886445703902287150], (-22498183352
16366117,-1756575569804098965], (-1756575569804098965,-1200694534887175029], (-36697626864585
08145,-3418773258395380497], (-5891139956868765777,-5295837587908366526], (-84164528188714983
54,-8180182878661376322], (-6215973879811251822,-5891139956868765777], (-7354100419201146814,
-711584188888312925], (2784664276580153761,3018280212442117293], (5576809590250259969,582906
9186973328933], (-2552524235586810551,-2249818335216366117], (-1200694534887175029,-789211326
725563467)] finished (progress: 57%)
[2026-06-08 20:31:33,108] Repair session 09a0ecd0-6379-11f1-8c9d-2ddff7a14c59 for range [(-67
08811889481116571,-6215973879811251822], (-5295837587908366526,-4815384075643186237], (-34187
73258395380497,-3000095575484690939], (9008140603904522259,-9078584325173739591], (2151680453
107019638,2419816521485378467], (3018280212442117293,3397961974001492543], (68864457039022871
50,7363935285752230219], (150856297475980269,519199489102755587], (-9078584325173739591,-8798
163910563576400], (4478867360398651050,4909878841329319133], (5829069186973328933,62726381750
60017960], (-8180182878661376322,-7726663294223543587], (7876804589620440680,8208311933243391
977], (7363935285752230219,7876804589620440680], (4909878841329319133,5176596274825079671], (-
4815384075643186237,-4384058150856169138], (-711584188888312925,-6708811889481116571], (175
985949646674205,2151680453107019638], (-4384058150856169138,-4079468382932292596)] finished
(progress: 57%)
[2026-06-08 20:31:33,141] Repair completed successfully
```

Slika 32- Izvršenje anty-entropy repair-a

Pregledom logova sa drugih čvorova može se videti proces koji se dešava prilikom anty-entropy repair-a, a deo logova je prikazan na slici 33.

```
2596]], hasSkippedReplicas=false) for keyspace_rf2.[users]
INFO [RepairJobTask:1] 2026-06-08T20:31:32,895 RepairJob.java:497 - [repair #099e06a0-6379-11f1-8c9d-2ddff7a14c59] Requesting merkle trees for users (to [/172.18.0.3:7000, /172.18.0.2:7000])
INFO [RepairJobTask:1] 2026-06-08T20:31:32,907 RepairJob.java:136 - [repair #09a0ecd0-6379-11f1-8c9d-2ddff7a14c59] keyspace_rf2.users not running paxos repair
INFO [RepairJobTask:1] 2026-06-08T20:31:32,907 RepairJob.java:497 - [repair #09a0ecd0-6379-11f1-8c9d-2ddff7a14c59] Requesting merkle trees for users (to [/172.18.0.4:7000, /172.18.0.2:7000])
INFO [RequestResponseStage-1] 2026-06-08T20:31:32,911 RepairMessage.java:259 - [09865ff0-6379-11f1-8c9d-2ddff7a14c59] VALIDATION_REQ received by /172.18.0.3:7000
INFO [RequestResponseStage-2] 2026-06-08T20:31:32,912 RepairMessage.java:259 - [09865ff0-6379-11f1-8c9d-2ddff7a14c59] VALIDATION_REQ received by /172.18.0.2:7000
INFO [RequestResponseStage-1] 2026-06-08T20:31:32,916 RepairMessage.java:259 - [09865ff0-6379-11f1-8c9d-2ddff7a14c59] VALIDATION_REQ received by /172.18.0.4:7000
INFO [RequestResponseStage-2] 2026-06-08T20:31:32,916 RepairMessage.java:259 - [09865ff0-6379-11f1-8c9d-2ddff7a14c59] VALIDATION_REQ received by /172.18.0.2:7000
INFO [AntiEntropyStage:1] 2026-06-08T20:31:32,940 RepairSession.java:242 - [repair #09a0ecd0-6379-11f1-8c9d-2ddff7a14c59] Received merkle tree for users from /172.18.0.4:7000
INFO [AntiEntropyStage:1] 2026-06-08T20:31:32,957 Validator.java:262 - [repair #099e06a0-6379-11f1-8c9d-2ddff7a14c59] Local completed merkle tree for /172.18.0.2:7000 for keyspace_rf2.users
INFO [AntiEntropyStage:1] 2026-06-08T20:31:32,961 RepairSession.java:242 - [repair #099e06a0-6379-11f1-8c9d-2ddff7a14c59] Received merkle tree for users from /172.18.0.2:7000
INFO [AntiEntropyStage:1] 2026-06-08T20:31:32,961 Validator.java:262 - [repair #09a0ecd0-6379-11f1-8c9d-2ddff7a14c59] Local completed merkle tree for /172.18.0.2:7000 for keyspace_rf2.users
INFO [AntiEntropyStage:1] 2026-06-08T20:31:32,963 RepairSession.java:242 - [repair #09a0ecd0-6379-11f1-8c9d-2ddff7a14c59] Received merkle tree for users from /172.18.0.2:7000
INFO [AntiEntropyStage:1] 2026-06-08T20:31:32,965 RepairSession.java:242 - [repair #099e06a0-6379-11f1-8c9d-2ddff7a14c59] Received merkle tree for users from /172.18.0.3:7000
INFO [RepairJobTask:2] 2026-06-08T20:31:32,965 RepairJob.java:358 - Created 0 sync tasks based on 2 merkle tree responses for 09865ff0-6379-11f1-8c9d-2ddff7a14c59 (took: 0ms)
INFO [RepairJobTask:3] 2026-06-08T20:31:32,965 RepairJob.java:358 - Created 1 sync tasks based on 2 merkle tree responses for 09865ff0-6379-11f1-8c9d-2ddff7a14c59 (took: 2ms)
INFO [RepairJobTask:2] 2026-06-08T20:31:32,965 RepairJob.java:209 - [repair #099e06a0-6379-11f1-8c9d-2ddff7a14c59] keyspace_rf2.users is fully synced
INFO [RepairJobTask:3] 2026-06-08T20:31:32,966 SyncTask.java:93 - [repair #09a0ecd0-6379-11f1-8c9d-2ddff7a14c59] Endpoints /172.18.0.2:7000 and /172.18.0.4:7000 have 19 range(s) out of sync for users
INFO [RepairJobTask:3] 2026-06-08T20:31:32,966 LocalSyncTask.java:118 - [repair #09a0ecd0-6379-11f1-8c9d-2ddff7a14c59] Performing streaming repair of 19 ranges with /172.18.0.4:7000
INFO [RepairJobTask:3] 2026-06-08T20:31:32,966 RepairSession.java:355 - [repair #099e06a0-6379-11f1-8c9d-2ddff7a14c59] Session completed successfully
```

Slika 33- Deo logova prilikom anty-entropy repair procesa



## 6 Literatura

- [1] Carpenter, Jeff, and Eben Hewitt. *Cassandra: The Definitive Guide*. “O’Reilly Media, Inc.,” 29 June 2016.
- [2] “Documentation.” *Cassandra.apache.org*, [cassandra.apache.org/doc/latest/](https://cassandra.apache.org/doc/latest/).
- [3] Kleppmann, Martin. *Designing Data-Intensive Applications : The Big Ideas behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, Ca, O’reilly Media, 2018.
- [4] IBM. “CAP Theorem.” *Ibm.com*, 20 Dec. 2022, [www.ibm.com/think/topics/cap-theorem](https://www.ibm.com/think/topics/cap-theorem).
- [5] Wikipedia Contributors. “Consistent Hashing.” *Wikipedia*, Wikimedia Foundation, 22 Sept. 2024, [en.wikipedia.org/wiki/Consistent\\_hashing](https://en.wikipedia.org/wiki/Consistent_hashing).
- [6] Lynch, Joseph, and Snyder Josh. *Cassandra Availability with Virtual Nodes*, 16 April 2018.